TP 2 · PROJET FIL ROUGE

Construis ton CRM de A à Z

React · Express · Node.js · MongoDB Atlas
Postman · Git · GitHub · Vercel

> **Ta mission :** une entreprise gère ses clients dans des fichiers Excel et perd des informations. Tu vas lui construire un vrai CRM, en ligne, utilisable depuis n'importe quel navigateur.

Professeur Othman Mekouar
Module : CRM / Développement web full-stack
Durée indicative : 6 à 8 heures

Nom : ..... Groupe : .....

La carte de l'aventure

Tu avances niveau par niveau. Chaque niveau se termine par un **point de contrôle** : ne passe pas au suivant tant qu'il n'est pas validé.

Niveau	Ce que tu fais	Ce que tu débloques
0	Comprendre le projet	CRM, frontend, backend, API, base de données
1	Installer les outils et créer les comptes	Node.js, VS Code, Git, Postman, GitHub, Atlas, Vercel
2	Créer le projet	Dossier, terminal, Git
3	Créer la base MongoDB Atlas	Cluster gratuit, utilisateur, IP autorisée, URI
4	Créer le premier serveur	Node.js, Express, npm, route
5	Construire l'API du CRM	Mongoose, modèle, routes CRUD, codes HTTP
6	Tester avec Postman	Requêtes, JSON, erreurs
7	Découvrir React	Create React App, SPA, JSX, composants, props, état
8	Construire l'interface du CRM	fetch, formulaire, tableau, recherche
9	Publier sur GitHub	commit, push, sécurité des secrets
10	Déployer sur Vercel	Backend et frontend en ligne, variables d'environnement
Boss final	Présenter ton CRM	Démonstration, questions, barème

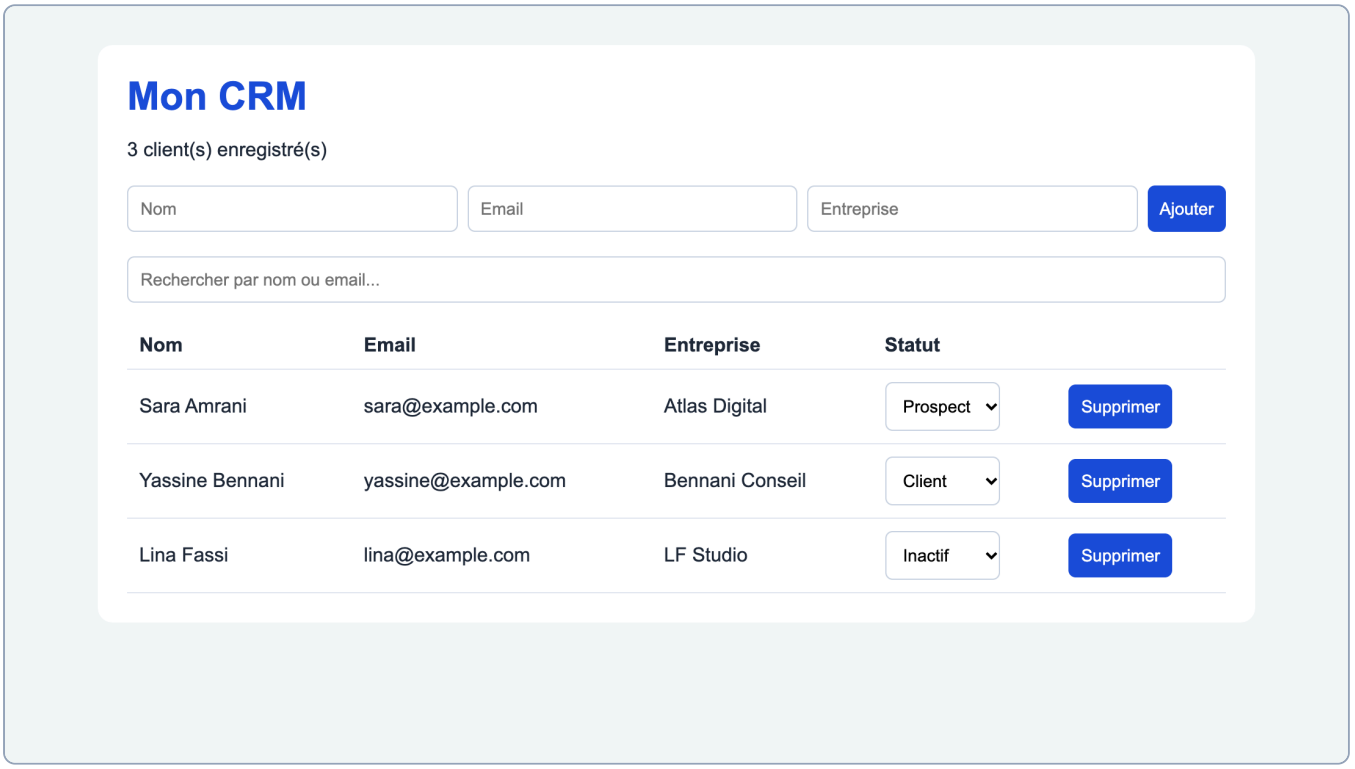
> **Comment lire les images.** Le badge CAPTURE RÉELLE indique une vraie capture faite pendant le test de ce TP. Le badge SCHÉMA indique un dessin explicatif. Les sites évoluent : si un bouton a légèrement changé de nom ou de place, cherche le mot le plus proche.

Niveau 0 — Comprendre le projet

C'est quoi un CRM ?

CRM veut dire **Customer Relationship Management** : gestion de la relation client. C'est un logiciel qui rassemble au même endroit les contacts d'une entreprise et leur suivi commercial. Notre CRM permettra de :

- ajouter un client (nom, email, entreprise) ;
- voir tous les clients dans un tableau ;
- changer son statut : **prospect**, **client** ou **inactif** ;
- supprimer un client ;
- rechercher par nom ou email ;
- retrouver toutes les données même après avoir fermé l'ordinateur.



CAPTURE RÉELLE Voici ce que tu auras construit à la fin du TP.

Les 3 parties d'une application web

SCHÉMA **L'architecture de notre CRM**



Compare avec un restaurant : le frontend est la salle et le menu, le backend est le serveur et la cuisine, la base de données est le garde-manger. Le client ne va jamais lui-même dans le garde-manger : il passe commande au serveur.

> **Règle d'or :** React ne parle jamais directement à MongoDB. Le navigateur appelle l'API, et seule l'API parle à la base. Sinon, n'importe qui pourrait lire le mot de passe de la base dans le navigateur.

MERN, ça veut dire quoi ?

Lettre	Technologie	Son rôle
M	MongoDB	La base de données (nous utilisons sa version cloud : Atlas)
E	Express	Crée les routes de l'API
R	React	Crée l'interface
N	Node.js	Fait tourner JavaScript en dehors du navigateur, sur le serveur

API, HTTP, JSON : les 3 mots à connaître

API : la liste des demandes qu'on a le droit de faire au serveur. **HTTP :** la langue utilisée pour faire ces demandes. **JSON :** le format des données échangées. Voici un client en JSON :

Un client au format JSON

```
{
  "nom": "Sara Amrani",
  "email": "sara@example.com",
  "entreprise": "Atlas Digital",
  "statut": "prospect"
}
```

Chaque demande HTTP a une **méthode** qui dit ce qu'on veut faire :

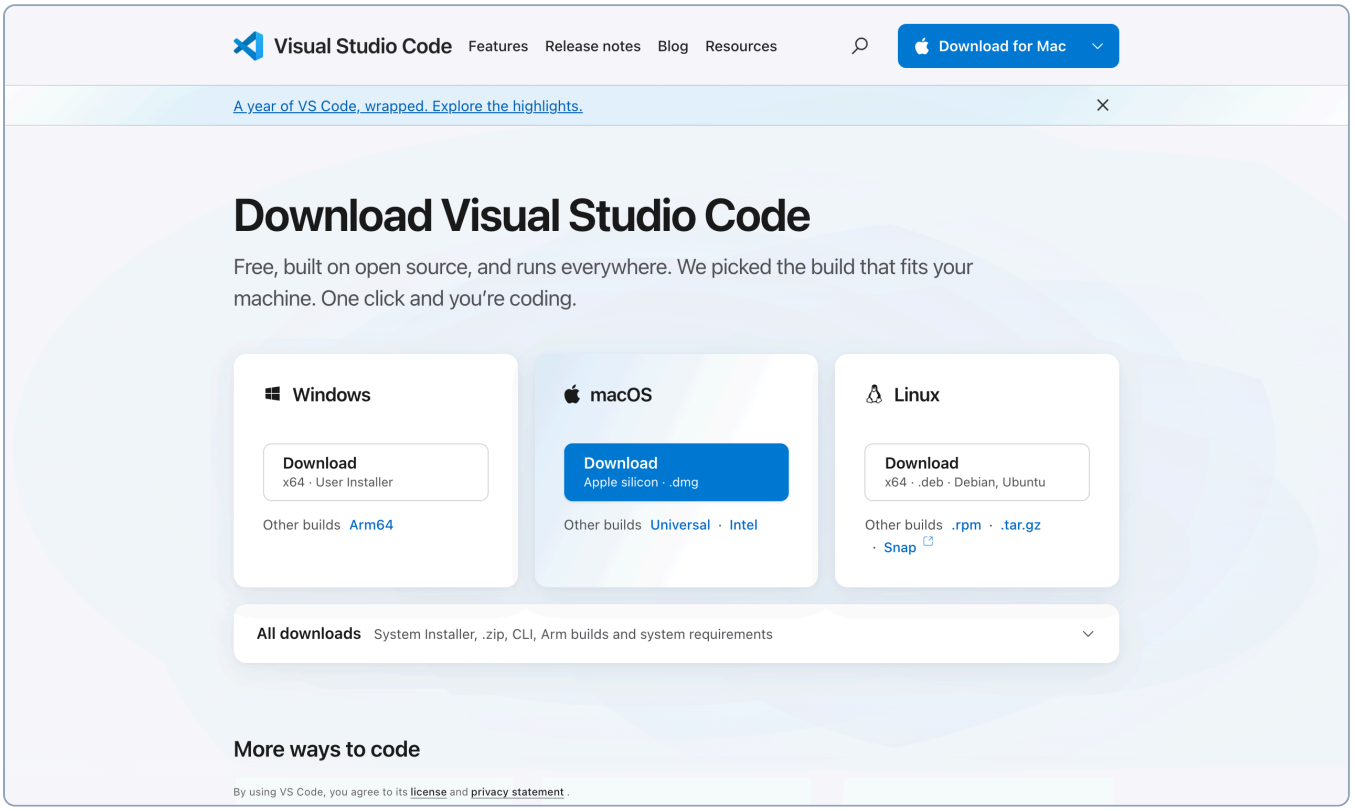
Méthode	Action	Adresse dans notre CRM	Succès
GET	Lire	/api/clients	200
GET	Lire un seul client	/api/clients/:id	200
POST	Créer	/api/clients	201
PATCH	Modifier	/api/clients/:id	200
DELETE	Supprimer	/api/clients/:id	200

Ces 4 actions s'appellent le **CRUD** : Create, Read, Update, Delete.

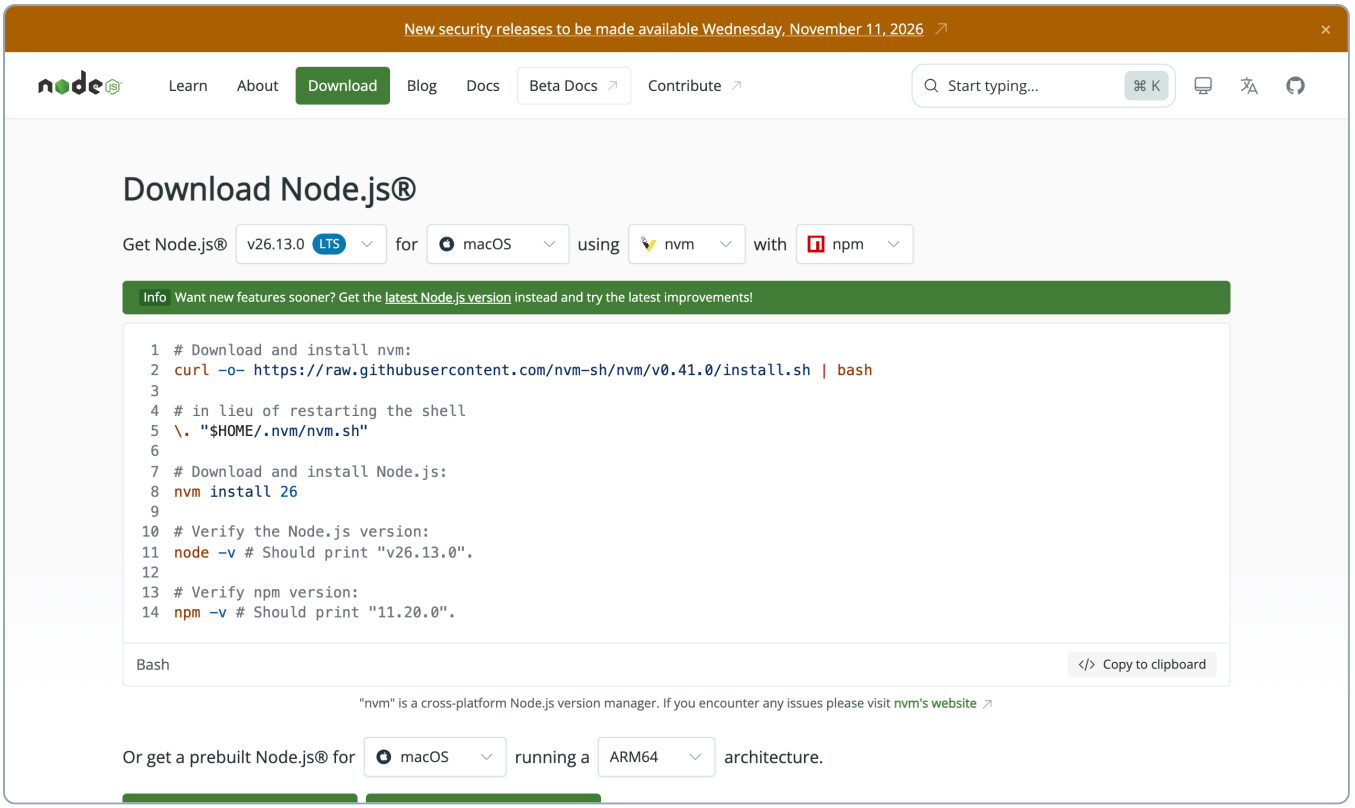
> ✅ **Point de contrôle :** explique à ton voisin, sans regarder, le rôle du frontend, du backend et de la base de données.

Niveau 1 — Installer les outils et créer les comptes

Outil	À quoi il sert	Où le trouver
Node.js (version LTS)	Exécuter JavaScript et installer des paquets avec npm	nodejs.org
Visual Studio Code	Écrire le code	code.visualstudio.com
Git	Garder l'historique du code	git-scm.com
Postman	Tester l'API sans interface	postman.com/downloads



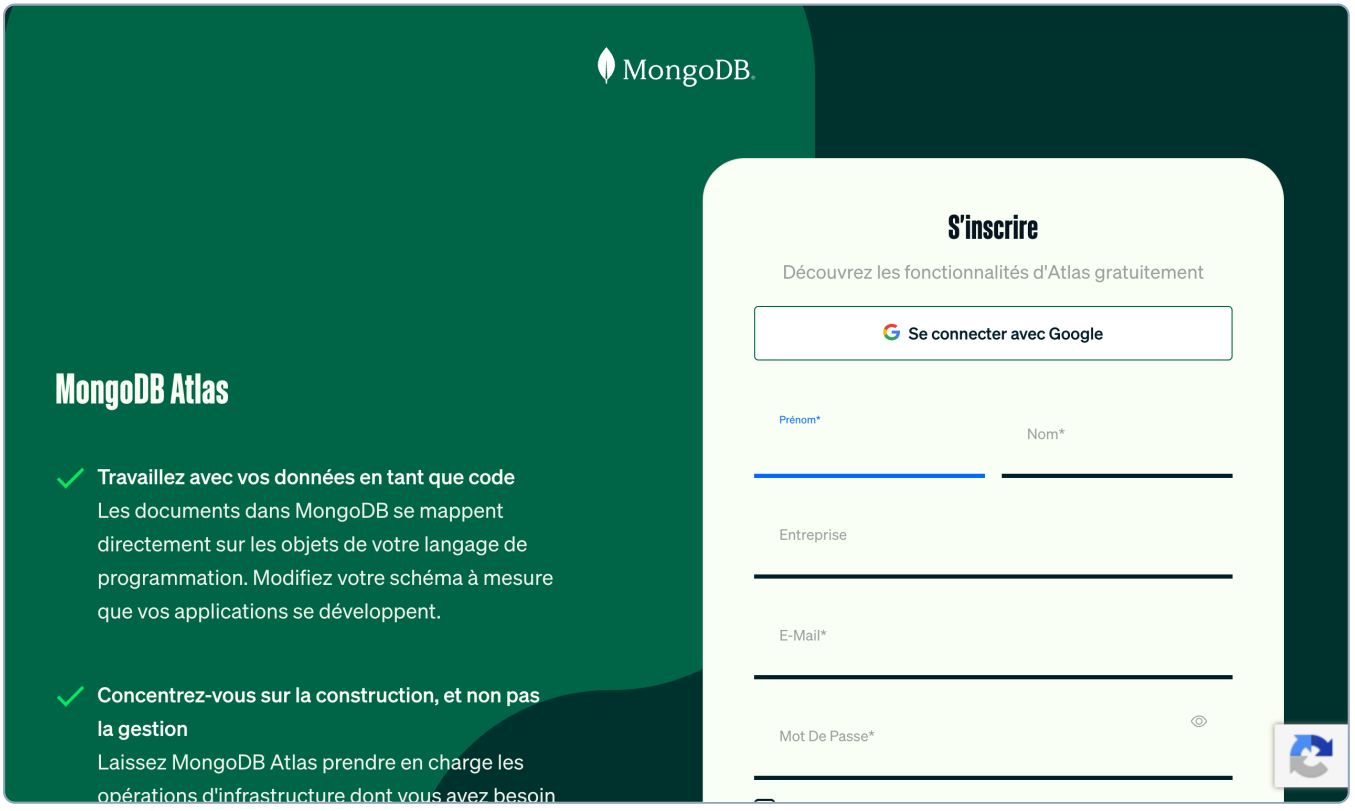
CAPTURE RÉELLE code.visualstudio.com : le site propose la version adaptée à ton ordinateur.



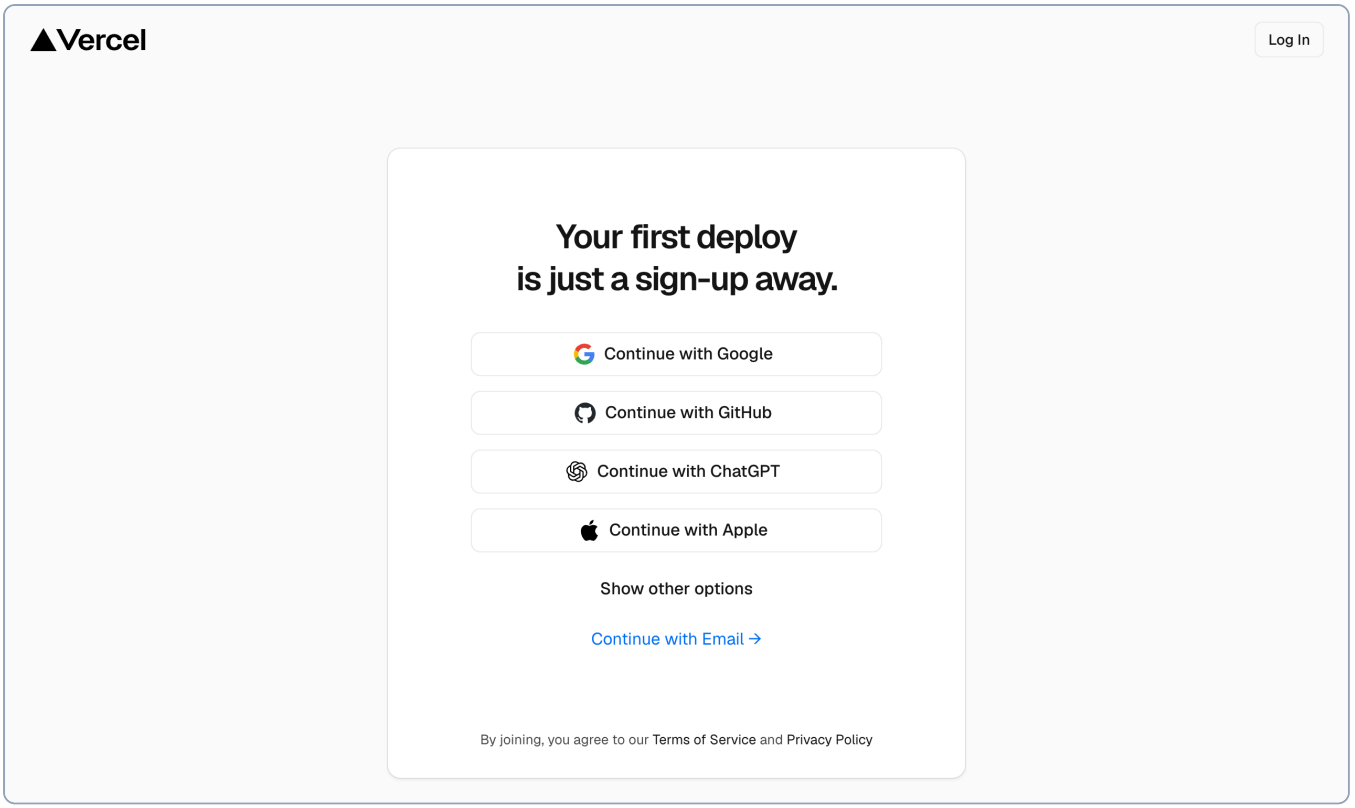
CAPTURE RÉELLE nodejs.org : choisis la version LTS. Sur Windows, prends l'installateur (.msi) plus bas sur la page.

Crée tes 3 comptes gratuits

1. **GitHub** sur github.com : il hébergera ton code.
2. **MongoDB Atlas** sur mongodb.com/cloud/atlas/register : il hébergera ta base.
3. **Vercel** sur vercel.com/signup : choisis **Continue with GitHub** pour relier les deux directement.



CAPTURE RÉELLE Page d'inscription à MongoDB Atlas.



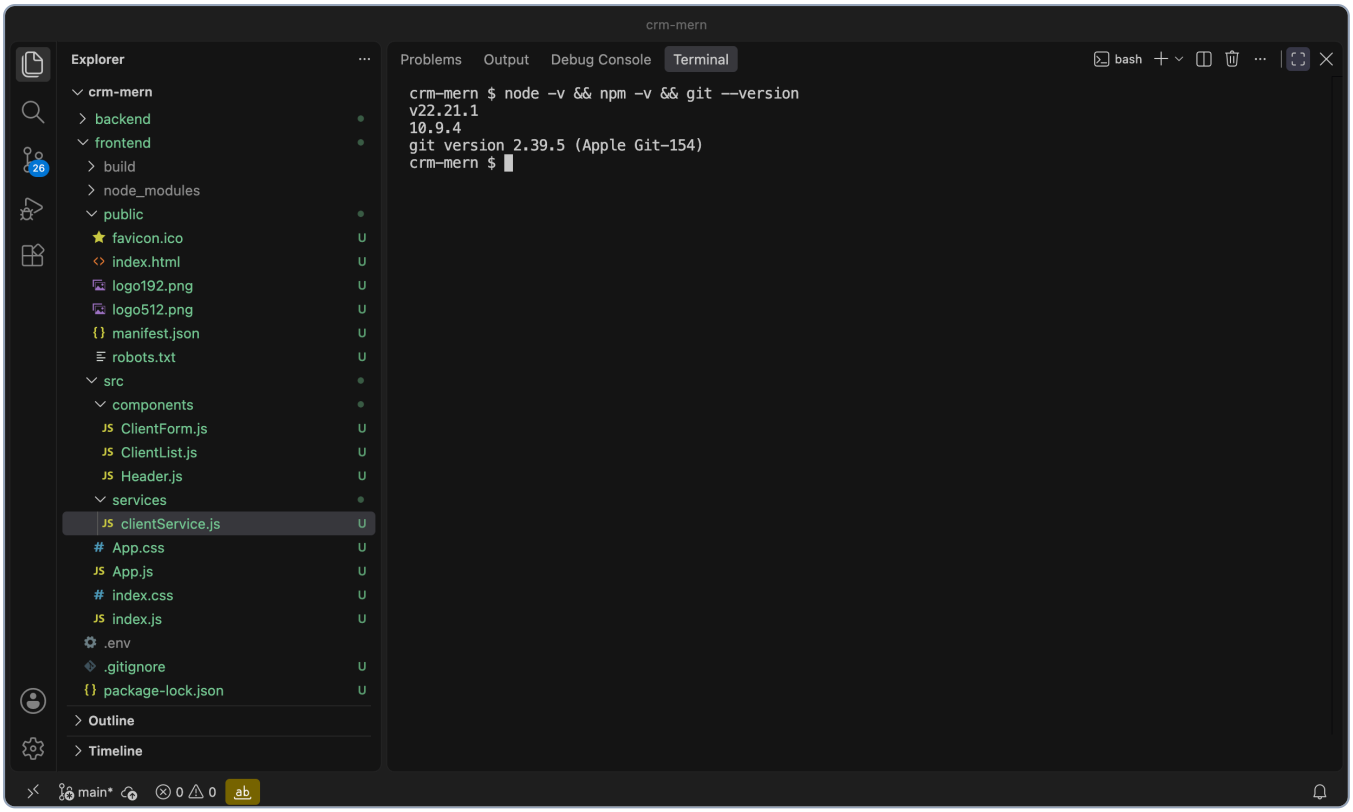
CAPTURE RÉELLE Page d'inscription à Vercel : utilise le bouton « Continue with GitHub ».

Vérifie les installations

Ouvre VS Code, puis le menu **Terminal > New Terminal**, et tape :

Terminal

```
node –v
npm –v
git ––version
```



CAPTURE RÉELLE Chaque commande répond par un numéro de version. Tes numéros peuvent être différents.

> **Si « command not found » s'affiche :** ferme puis rouvre VS Code. Si cela ne suffit pas, réinstalle l'outil concerné.

> ✅ **Point de contrôle :** les 3 commandes répondent et tes 3 comptes sont créés.

Niveau 2 — Créer le projet

Étape 1 : créer et ouvrir le dossier

Crée un dossier **crm-mern** sur ton bureau. Dans VS Code : **File > Open Folder**, puis sélectionne ce dossier. Ouvre ensuite le terminal : il est maintenant placé dans **crm-mern**.

Étape 2 : lancer Git tout de suite

Terminal — dans crm-mern

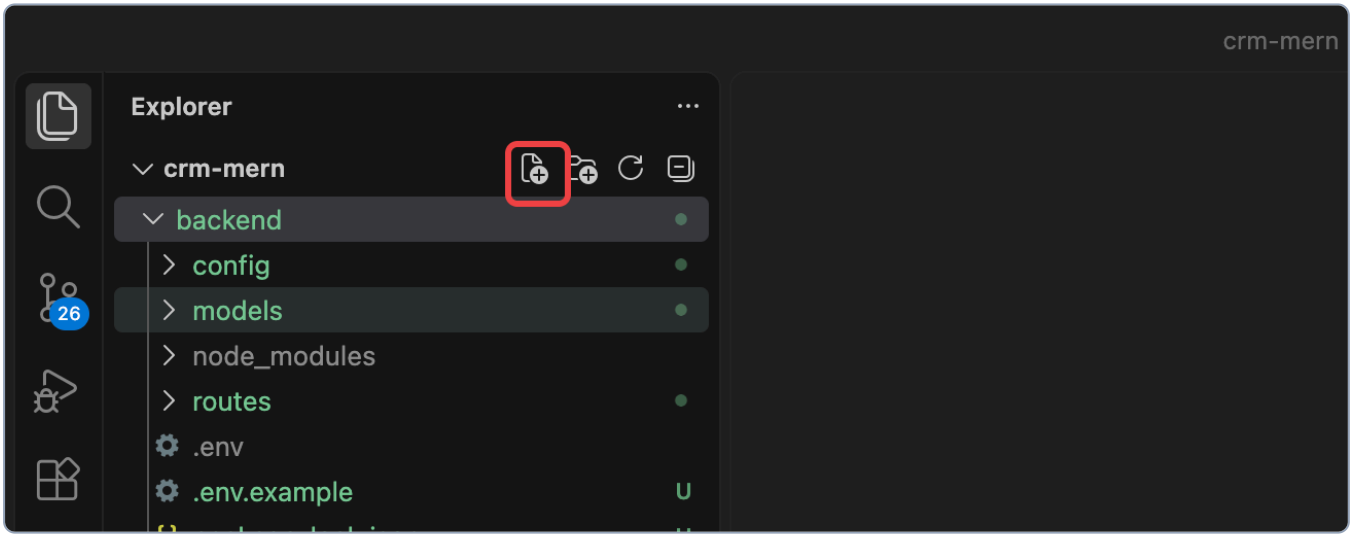
```
git init -b main
```

Cette commande transforme le dossier en dépôt Git, c'est-à-dire qu'il pourra garder l'historique de tes modifications.

> **Piège important :** fais `git init` avant de créer le frontend. Sinon, Create React App crée son propre dépôt Git dans **frontend**, et GitHub affichera ce dossier vide. Si cela t'arrive, supprime le dossier caché `frontend/.git`.

Étape 3 : dire à Git ce qu'il doit ignorer

Dans l'explorateur de VS Code, clique sur l'icône **New File** et crée le fichier **.gitignore** à la racine.



CAPTURE RÉELLE Le bouton New File (encadré en rouge). Clique d'abord sur un dossier pour créer le fichier à l'intérieur.

.gitignore

```
node_modules/
.env
build/
.vercel/
.DS_Store
```

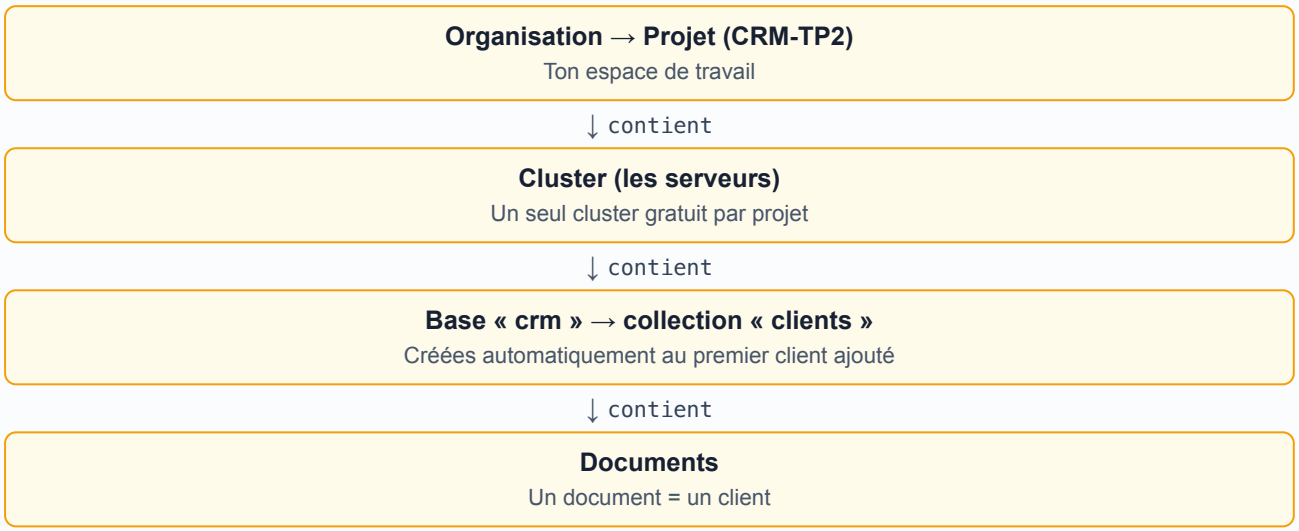
Ligne	Pourquoi l'ignorer ?
node_modules/	Dossier énorme, recréé à tout moment avec `npm install`
.env	Contient tes mots de passe : **il ne doit jamais aller sur GitHub**
build/	Fichiers générés automatiquement
.vercel/, .DS_Store	Fichiers techniques inutiles dans le dépôt

✅ **Point de contrôle :** VS Code est ouvert sur **crm-mern**, et `git status` répond sans erreur.

Niveau 3 — Créer la base MongoDB Atlas

MongoDB ne range pas les données dans des tableaux, mais dans des **documents** qui ressemblent à du JSON. Les documents sont regroupés dans des **collections**, elles-mêmes regroupées dans une **base**. **Atlas** est le service qui héberge MongoDB dans le cloud : rien à installer sur ton ordinateur.

SCHÉMA **Comment Atlas est organisé**



> **Et MongoDB Compass ?** Compass est une application à installer pour voir ses données depuis son ordinateur. Elle sert surtout quand MongoDB tourne en **local**, sur ta machine. **Dans ce TP, on ne l'utilise pas** : on travaille directement sur Atlas, et on regarde les données dans son **Data Explorer**. Il est simplement utile de savoir que cet outil existe.

Étape 1 : créer un projet

1. Connecte-toi sur cloud.mongodb.com.
2. Crée un nouveau projet (**New Project**) et nomme-le **CRM-TP2**.
3. Valide avec **Create Project**.

Étape 2 : créer le cluster gratuit

1. Dans **Project Overview**, clique sur **Create**.
2. Choisis l'option gratuite **Free (M0)**.
3. Choisis un fournisseur, par exemple **AWS**, et la région la plus proche, par exemple en Europe.
4. Dans **Name**, écris **crm-cluster**. Attention : ce nom ne pourra plus être changé.
5. Clique sur **Create**. Le cluster se prépare en quelques instants.

Le cluster gratuit, ce qu'il faut savoir	
Prix	Gratuit, sans limite de durée
Nombre	**Un seul cluster gratuit par projet.** Si Atlas refuse d'en créer un autre, c'est normal : réutilise le tien
Stockage	0,5 Go : largement assez pour ce TP
Mise en pause	Après 30 jours sans aucune connexion. Tu pourras le relancer

Étape 3 : créer l'utilisateur de la base

> **Ne confonds pas :** ton compte Atlas (email et mot de passe pour aller sur le site) et l'**utilisateur de base de données** (identifiant et mot de passe utilisés par ton code). Ce sont deux choses différentes.

1. Dans le menu de gauche, section **Security**, clique sur **Database & Network Access**.
2. Onglet **Database Users**, puis **Add New Database User**.
3. Méthode : **Password**.
4. Identifiant : **crm_user**. Mot de passe : choisis-en un long, **avec seulement des lettres et des chiffres**, et note-le.
5. Dans **Built-in Role**, choisis le rôle de lecture et d'écriture sur les bases (**Read and write to any database**).
6. Clique sur **Add User**.

> **Pourquoi seulement des lettres et des chiffres ?** Le mot de passe sera écrit dans une adresse (URI). Des caractères comme `@ / : ? #` y ont un sens spécial et provoquent des erreurs, à moins de les encoder (`@` devient `%40`).

Étape 4 : autoriser les adresses IP

Par défaut, Atlas refuse toutes les connexions. Il faut dire quelles adresses ont le droit de se connecter : c'est l'**IP Access List**.

1. Toujours dans **Database & Network Access**, ouvre **IP Access List**.
2. Clique sur **Add IP Address**.
3. Pour ce TP, saisis **0.0.0.0/0** (accès depuis n'importe où), ajoute le commentaire « TP CRM », puis clique sur **Confirm**.

> **Pourquoi 0.0.0.0/0 ?** Vercel n'a pas une adresse IP fixe : sans cela, ton API en ligne ne pourrait pas se connecter. **Ce réglage a un risque :** seul le mot de passe protège alors ta base. Atlas envoie d'ailleurs un email d'alerte, c'est normal. Utilise donc un mot de passe solide et uniquement des données fictives. Dans une vraie entreprise, on restreint cet accès.

Étape 5 : récupérer l'adresse de connexion (URI)

1. Dans le menu, section **Database**, clique sur **Clusters**.
2. Sur ton cluster, clique sur **Connect**.

3. Choisis **Drivers**, puis **Node.js**.
4. Copie l'adresse qui commence par `mongodb+srv://`.

Elle ressemble à ceci :

L'adresse copiée depuis Atlas

```
mongodb+srv://crm_user:<db_password>@crm-cluster.xxxxx.mongodb.net/?
retryWrites=true&w=majority
```

Fais **deux modifications** : remplace `<db_password>` par ton mot de passe (sans les chevrons), et ajoute le nom de la base, **crm**, juste avant le `?` :

L'adresse finale à garder

```
mongodb+srv://crm_user:MonMotDePasse123@crm-cluster.xxxxx.mongodb.net/crm?
retryWrites=true&w=majority
```

SCHÉMA **Les morceaux de l'URI**

crm_user : MonMotDePasse123
L'utilisateur de la base et son mot de passe

crm-cluster.xxxxx.mongodb.net
L'adresse de ton cluster

/crm
Le nom de la base. Sans lui, tes données iraient dans une base nommée « test »

Cette adresse est un secret. Quiconque la possède peut lire et effacer ta base. Ne la mets jamais dans ton code, sur GitHub ou dans une capture d'écran.

✅ **Point de contrôle :** tu as un cluster gratuit, un utilisateur de base, une IP autorisée et ton URI complète, gardée de côté.

Niveau 4 — Créer le premier serveur

Étape 1 : préparer le dossier backend

Terminal — dans crm-mern

```
mkdir backend
cd backend
npm init -y
npm install express mongoose cors dotenv
npm install --save-dev nodemon
```

Paquet	Son rôle
express	Créer le serveur et les routes
mongoose	Parler à MongoDB simplement
cors	Autoriser le frontend à appeler l'API
dotenv	Lire le fichier .env
nodemon	Redémarrer le serveur à chaque enregistrement (seulement pendant le développement)

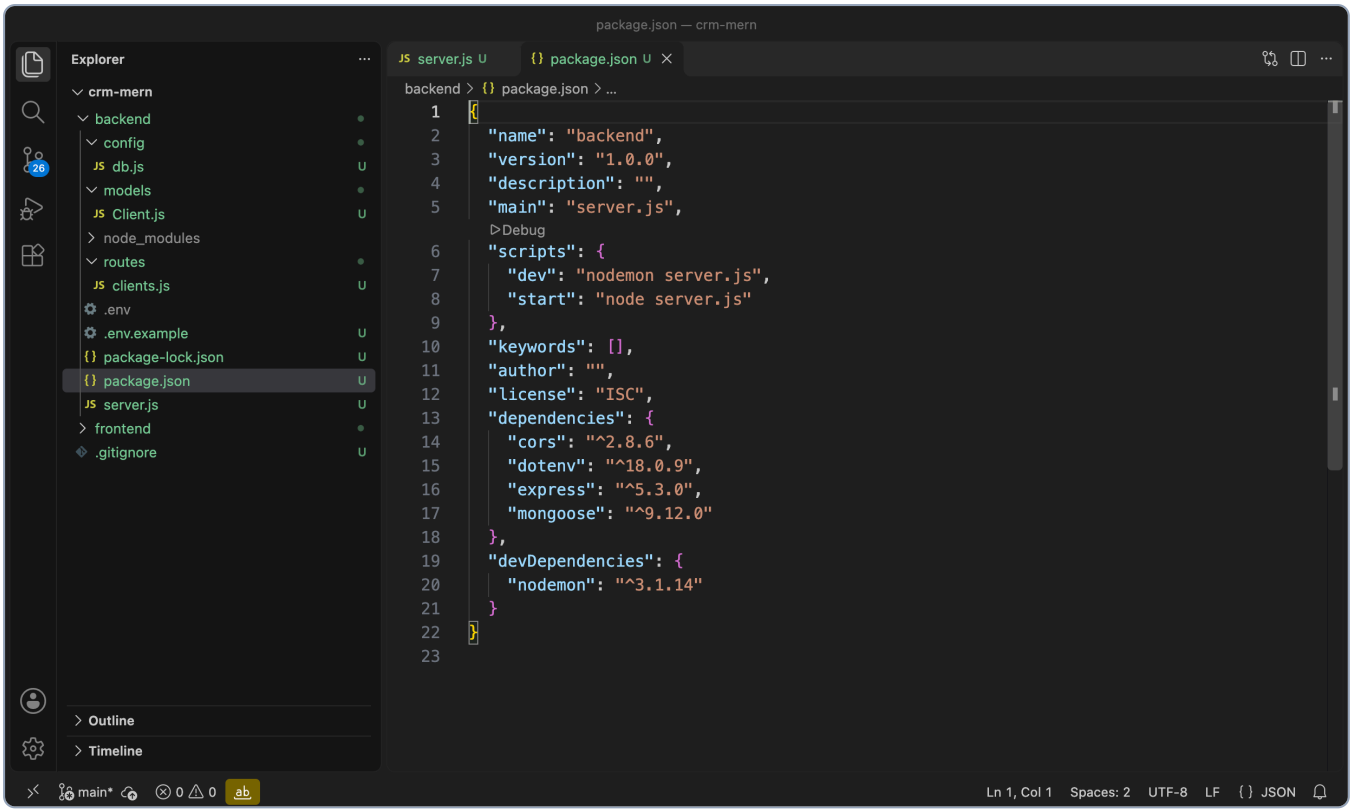
Trois choses sont apparues : **package.json** (la carte d'identité du projet), **package-lock.json** (les versions exactes) et **node_modules** (le code des paquets).

Étape 2 : ajouter les commandes

Ouvre **backend/package.json** et remplace le bloc `"scripts"` par :

backend/package.json — extrait

```
"scripts": {
  "dev": "nodemon server.js",
  "start": "node server.js"
},
```



CAPTURE RÉELLE Le fichier package.json dans VS Code, avec les deux commandes. Enregistre avec Ctrl+S (Cmd+S sur Mac).

Étape 3 : écrire le plus petit serveur possible

Crée le fichier **backend/server.js** :

backend/server.js — première version

```
const express = require("express");

const app = express();

app.get("/", (req, res) => {
  res.json({ message: "API CRM opérationnelle" });
});

app.listen(4000, () => {
  console.log("Serveur démarré sur http://localhost:4000");
});
```

Ligne	Ce qu'elle fait
`require("express")`	Charge le paquet Express
`const app = express()`	Crée l'application
`app.get("/", ...)`	Quand quelqu'un demande l'adresse **/** en GET, exécute cette fonction
`req` / `res`	**req** : la demande reçue. **res** : la réponse à envoyer
`res.json(...)`	Répond avec du JSON
`app.listen(4000, ...)`	Démarre le serveur et attend les demandes sur le port 4000

Étape 4 : lancer et tester

Terminal — dans backend

```
npm run dev
```

Ouvre **http://localhost:4000** dans ton navigateur.

Impression élégante ☐

{"message":"API CRM opérationnelle"}

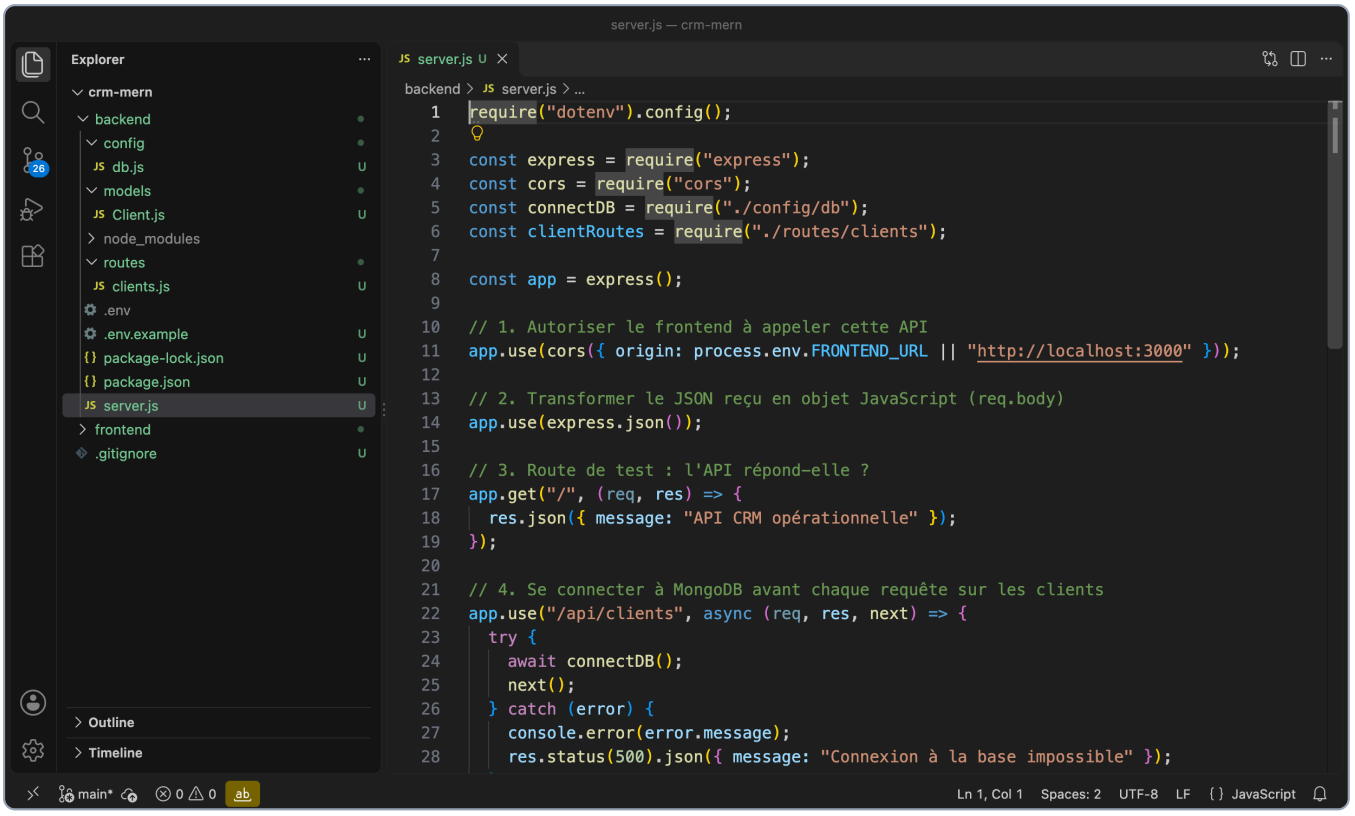
CAPTURE RÉELLE Ton serveur répond en JSON dans le navigateur. La version finale affichera le même message.

localhost veut dire « mon propre ordinateur ». **4000** est le numéro de la « porte » sur laquelle le serveur écoute. Le serveur tourne tant que le terminal reste ouvert : pour l'arrêter, fais **Ctrl+C**.

✅ **Point de contrôle :** le navigateur affiche le message JSON. Change le message dans le code, enregistre et actualise : nodemon a redémarré tout seul.

Niveau 5 — Construire l'API du CRM

Nous allons créer 4 fichiers. Voici leur organisation :



CAPTURE RÉELLE Le backend terminé dans VS Code. Le fichier .env est grisé : Git l'ignore bien.

SCHÉMA Le trajet d'une demande dans le backend

server.js
Reçoit la demande et l'envoie vers la bonne route

↓

routes/clients.js
Décide quoi faire selon la méthode et l'adresse

↓

models/Client.js
Vérifie les règles d'un client

↓

MongoDB Atlas
Enregistre ou renvoie les données

Étape 1 : mettre le secret dans .env

Crée **backend/.env** et colle ton URI Atlas à la place de l'exemple. Pas d'espaces autour du signe `=`.

backend/.env

```
PORT=4000
MONGODB_URI=mongodb+srv://UTILISATEUR:MOT_DE_PASSE@cluster.xxxxx.mongodb.net/crm
FRONTEND_URL=http://localhost:3000
```

Crée aussi **backend/.env.example** avec le même contenu, mais **sans ton vrai mot de passe**. Ce fichier ira sur GitHub pour montrer quelles variables sont nécessaires.

Étape 2 : la connexion à la base

Crée le dossier **config**, puis le fichier **backend/config/db.js** :

backend/config/db.js

```
const mongoose = require("mongoose");

async function connectDB() {
  // Déjà connecté ? Alors on ne fait rien.
  if (mongoose.connection.readyState === 1) {
    return;
  }

  await mongoose.connect(process.env.MONGODB_URI);
  console.log("MongoDB connecté");
}

module.exports = connectDB;
```

`async` et `await` servent à attendre une opération qui prend du temps, comme une connexion à Internet. `process.env.MONGODB_URI` lit la valeur écrite dans **.env**.

Étape 3 : le modèle Client

Le **modèle** décrit à quoi ressemble un client et ses règles. Crée **backend/models/Client.js** :

backend/models/Client.js

```
const mongoose = require("mongoose");

const clientSchema = new mongoose.Schema(
  {
    nom: {
      type: String,
      required: [true, "Le nom est obligatoire"],
      trim: true
    },
    email: {
      type: String,
      required: [true, "L'email est obligatoire"],
```

```
      unique: true,
      lowercase: true,
      trim: true
    },
    telephone: { type: String, trim: true },
    entreprise: { type: String, trim: true },
    statut: {
      type: String,
      enum: {
        values: ["prospect", "client", "inactif"],
        message: "Statut invalide : prospect, client ou inactif"
      },
      default: "prospect"
    }
  },
  { timestamps: true }
);

module.exports = mongoose.model("Client", clientSchema);
```

Option	Signification
required	Le champ est obligatoire, avec un message d'erreur
unique	Deux clients ne peuvent pas avoir le même email
trim	Enlève les espaces au début et à la fin
lowercase	Met le texte en minuscules
enum	Seules ces valeurs sont acceptées
default	Valeur utilisée si rien n'est envoyé
timestamps	Ajoute automatiquement les dates de création et de modification

MongoDB ajoute aussi tout seul un identifiant unique à chaque client : **_id**.

Étape 4 : les routes

Crée **backend/routes/clients.js**. C'est le cœur de l'API.

backend/routes/clients.js

```
const express = require("express");
const Client = require("../models/Client");

const router = express.Router();

// Une seule fonction pour transformer une erreur en réponse HTTP
function envoyerErreur(res, error) {
  if (error.code === 11000) {
```

```
    return res.status(409).json({ message: "Cet email existe déjà" });
  }
  if (error.name === "ValidationError") {
    return res.status(400).json({ message: error.message });
  }
  if (error.name === "CastError") {
    return res.status(400).json({ message: "Identifiant invalide" });
  }
  console.error(error);
  return res.status(500).json({ message: "Erreur du serveur" });
}

// GET /api/clients : lire tous les clients
router.get("/", async (req, res) => {
  try {
    const clients = await Client.find().sort({ createdAt: -1 });
    res.status(200).json(clients);
  } catch (error) {
    envoyerErreur(res, error);
  }
});

// GET /api/clients/:id : lire un seul client
router.get("/:id", async (req, res) => {
  try {
    const client = await Client.findById(req.params.id);
    if (!client) {
      return res.status(404).json({ message: "Client introuvable" });
    }
    res.status(200).json(client);
  } catch (error) {
    envoyerErreur(res, error);
  }
});

// POST /api/clients : créer un client
router.post("/", async (req, res) => {
  try {
    const client = await Client.create(req.body);
    res.status(201).json(client);
  } catch (error) {
    envoyerErreur(res, error);
  }
});

// PATCH /api/clients/:id : modifier un client
router.patch("/:id", async (req, res) => {
```

```
  try {
    const client = await Client.findByIdAndUpdate(req.params.id, req.body, {
      returnDocument: "after", // renvoyer le client APRÈS modification
      runValidators: true      // appliquer aussi les règles du modèle
    });
    if (!client) {
      return res.status(404).json({ message: "Client introuvable" });
    }
    res.status(200).json(client);
  } catch (error) {
    envoyerErreur(res, error);
  }
});

// DELETE /api/clients/:id : supprimer un client
router.delete("/:id", async (req, res) => {
  try {
    const client = await Client.findByIdAndDelete(req.params.id);
    if (!client) {
      return res.status(404).json({ message: "Client introuvable" });
    }
    res.status(200).json({ message: "Client supprimé" });
  } catch (error) {
    envoyerErreur(res, error);
  }
});

module.exports = router;
```

Comprendre ce fichier

Élément	Où se trouve l'information	Exemple
`req.body`	Dans le corps de la demande	Les données JSON d'un nouveau client
`req.params.id`	Dans l'adresse	/api/clients/**65ab12**

Fonction Mongoose	Ce qu'elle fait
`Client.find()`	Trouve tous les clients
`Client.findById(id)`	Trouve un client
`Client.create(data)`	Crée un client
`Client.findByIdAndUpdate(...)`	Modifie un client
`Client.findByIdAndDelete(id)`	Supprime un client

`try / catch` veut dire : « essaie ceci, et si une erreur arrive, ne plante pas, va dans **catch** ». La fonction `envoyerErreur` choisit alors le bon code HTTP :

Code	Signification	Dans notre CRM
200	OK	Lecture, modification ou suppression réussie
201	Créé	Client ajouté
400	Demande incorrecte	Champ manquant, statut ou identifiant invalide
404	Introuvable	Ce client n'existe pas
409	Conflit	Cet email existe déjà
500	Erreur serveur	Problème inattendu côté serveur

Étape 5 : le serveur complet

Remplace **tout** le contenu de **backend/server.js** :

backend/server.js — version finale

```
require("dotenv").config();

const express = require("express");
const cors = require("cors");
const connectDB = require("./config/db");
const clientRoutes = require("./routes/clients");

const app = express();

// 1. Autoriser le frontend à appeler cette API
app.use(cors({ origin: process.env.FRONTEND_URL || "http://localhost:3000" }));

// 2. Transformer le JSON reçu en objet JavaScript (req.body)
app.use(express.json());

// 3. Route de test : l'API répond-elle ?
app.get("/", (req, res) => {
  res.json({ message: "API CRM opérationnelle" });
});

// 4. Se connecter à MongoDB avant chaque requête sur les clients
app.use("/api/clients", async (req, res, next) => {
  try {
    await connectDB();
    next();
  } catch (error) {
    console.error(error.message);
    res.status(500).json({ message: "Connexion à la base impossible" });
  }
});

// 5. Toutes les routes des clients
```

```
app.use("/api/clients", clientRoutes);

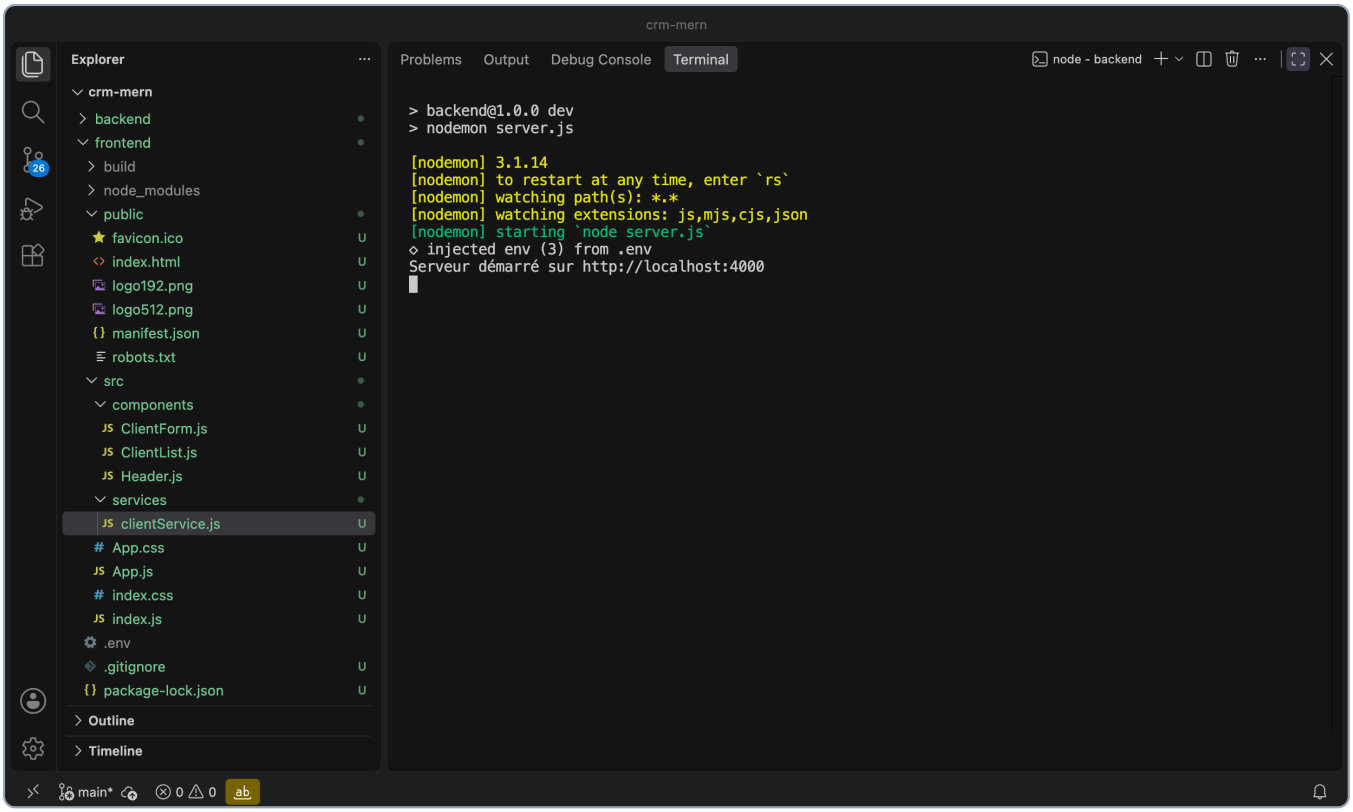
// 6. Dernier filet de sécurité (exemple : JSON mal écrit)
app.use((error, req, res, next) => {
  res.status(error.status || 500).json({ message: "Requête invalide" });
});

// 7. En local, on démarre le serveur. Sur Vercel, c'est Vercel qui s'en charge.
if (!process.env.VERCEL) {
  const PORT = process.env.PORT || 4000;
  app.listen(PORT, () => {
    console.log(`Serveur démarré sur http://localhost:${PORT}`);
  });
}

module.exports = app;
```

Partie	Explication
1. cors	Par sécurité, le navigateur bloque les appels vers un autre site. Ici, on autorise notre frontend
2. express.json()	Sans cette ligne, `req.body` serait vide
4. connectDB	Vérifie la connexion à la base avant de traiter les routes
7. process.env.VERCEL	En local on lance le serveur nous-mêmes. Sur Vercel, c'est Vercel qui utilise `module.exports = app`

Relance le serveur si besoin (`npm run dev`), puis ouvre **http://localhost:4000/api/clients**.



CAPTURE RÉELLE Le terminal indique que le serveur est démarré.

Une erreur « Connexion à la base impossible » ? Vérifie, dans l'ordre : l'URI dans .env, le mot de passe, **0.0.0.0/0** dans IP Access List, puis redémarre le serveur après chaque modification de .env.

✅ **Point de contrôle :** l'adresse /api/clients affiche `[]`, c'est-à-dire une liste vide. C'est normal : il n'y a pas encore de client.

Niveau 6 — Tester avec Postman

Le navigateur sait facilement faire un GET, mais pas un POST ou un DELETE. **Postman** permet d'envoyer n'importe quelle demande à l'API, avant même d'avoir l'interface.

> **Méthode de travail :** si l'API ne fonctionne pas dans Postman, ce n'est pas la peine de coder le frontend. On teste toujours l'API d'abord.

Laisse `npm run dev` tourner et ouvre Postman. Sans compte, clique sur **Continue without an account**, puis **Open Lightweight API Client**.

Test 1 : créer un client (POST)

1. Change la méthode **GET** en **POST** (liste déroulante à gauche).
2. Adresse : `http://localhost:4000/api/clients`.
3. Onglet **Body**, coche **raw**, puis choisis **JSON** dans la liste à droite.
4. Colle le JSON ci-dessous, puis clique sur **Send**.

Body de la requête

```
{
  "nom": "Nora El Idrissi",
  "email": "nora@example.com",
  "entreprise": "Nova Conseil"
}
```

Réponse de l'API — 201 Created

```
{
  "nom": "Nora El Idrissi",
  "email": "nora@example.com",
  "entreprise": "Nova Conseil",
  "statut": "prospect",
  "_id": "6aec4083ba8839b3963ff40f",
  "createdAt": "2026-11-05T08:38:27.479Z",
  "updatedAt": "2026-11-05T08:38:27.479Z",
  "__v": 0
}
```

MongoDB a ajouté _id, le statut par défaut et les dates.

Comment lire l'écran de Postman : en haut, ta demande. En bas, la réponse du serveur. Regarde d'abord le code affiché à droite (201 Created, par exemple), puis le JSON.

Test 2 : lire les clients (GET)

Méthode **GET**, même adresse, puis **Send**.

Réponse de l'API — 200 OK

```
[
  {
    "_id": "6aec4083ba8839b3963ff40f",
    "nom": "Nora El Idrissi",
    "email": "nora@example.com",
    "statut": "prospect"
  }
]
```

Une liste entre crochets [ ] : ici un seul client, résumé pour la lecture.

Ajoute 2 ou 3 clients pour avoir de quoi tester. **Copie la valeur d'un _id** : elle servira pour la suite.

Test 3 : modifier un client (PATCH)

Méthode **PATCH**. Adresse : `http://localhost:4000/api/clients/` suivie de l'_id copié. Body :

Body de la requête

```
{
  "statut": "client"
}
```

Réponse de l'API — 200 OK

```
{
  "_id": "6aec4083ba8839b3963ff40f",
  "nom": "Nora El Idrissi",
  "email": "nora@example.com",
  "statut": "client"
}
```

Le statut est passé à « client » (réponse résumée pour la lecture).

Test 4 : provoquer les erreurs exprès

Un bon développeur vérifie aussi ce qui se passe quand l'utilisateur se trompe.

Test	Comment faire	Attendu
Email en double	POST avec un email déjà utilisé	**409**
Champ manquant	POST avec seulement un email	**400**
Statut interdit	PATCH avec `{"statut": "urgent"}`	**400**
Identifiant invalide	GET /api/clients/bonjour	**400**
Client inexistant	DELETE deux fois de suite sur le même _id	**404**

Réponse de l'API — 409 Conflict

```
{
  "message": "Cet email existe déjà"
}
```

Le serveur ne plante pas et explique le problème.

Réponse de l'API — 400 Bad Request

```
{
  "message": "Client validation failed: nom: Le nom est obligatoire"
}
```

C'est le message écrit dans notre modèle.

Test 5 : supprimer un client (DELETE)

Méthode **DELETE**, adresse avec l'_id, puis **Send**. Réponse attendue : **200** et le message `Client supprimé`.

Test 6 : vérifier dans Atlas

1. Sur Atlas, va dans **Clusters**, puis ouvre **Data Explorer**.
2. Ouvre la base **crm**, puis la collection **clients**.
3. Tes clients sont là : ils sont donc bien enregistrés dans le cloud.

> ✅ **Point de contrôle :** tu as obtenu les codes 200, 201, 400, 404 et 409, et tu vois tes clients dans le Data Explorer d'Atlas. **Ton backend est terminé !** Garde son terminal ouvert.

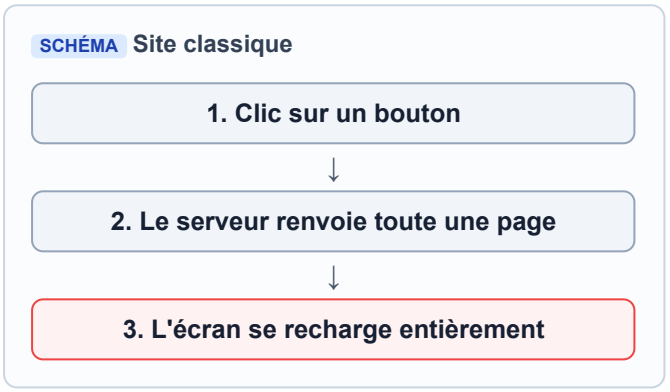
Niveau 7 — Découvrir React

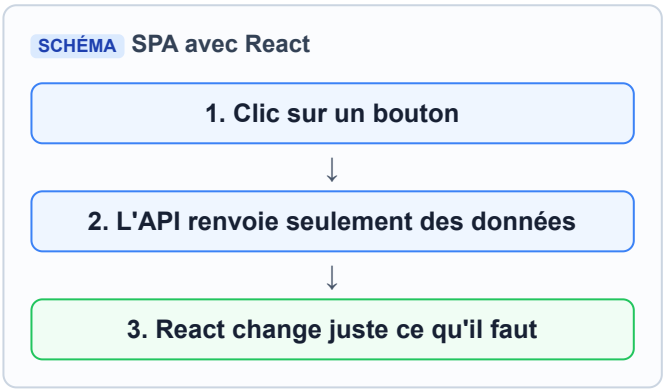
C'est quoi React ?

React est une bibliothèque JavaScript pour construire des interfaces. Au lieu d'écrire une grande page, on l'assemble avec de petites pièces réutilisables, comme des Lego : les **composants**.

C'est quoi une SPA ?

SPA veut dire **Single Page Application**, une application en une seule page. Le serveur envoie une seule page HTML presque vide, puis JavaScript construit et met à jour l'écran directement dans le navigateur.





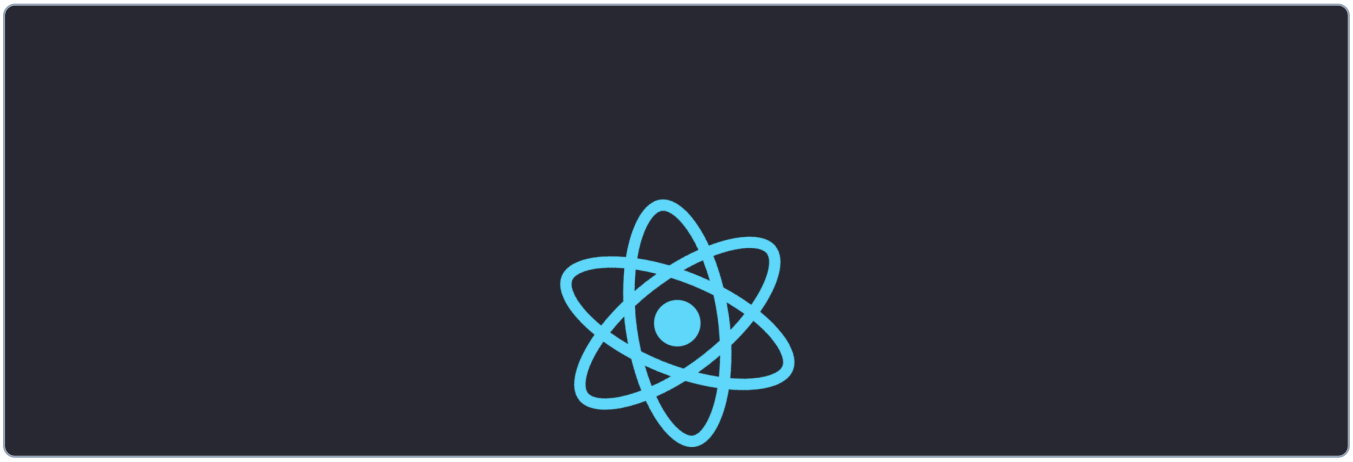
Étape 1 : créer l'application

Ouvre un **deuxième terminal** (bouton **+** du terminal) pour laisser le backend tourner, et place-toi à la racine :

Nouveau terminal — dans crm-mern

```
npx create-react-app frontend
cd frontend
npm start
```

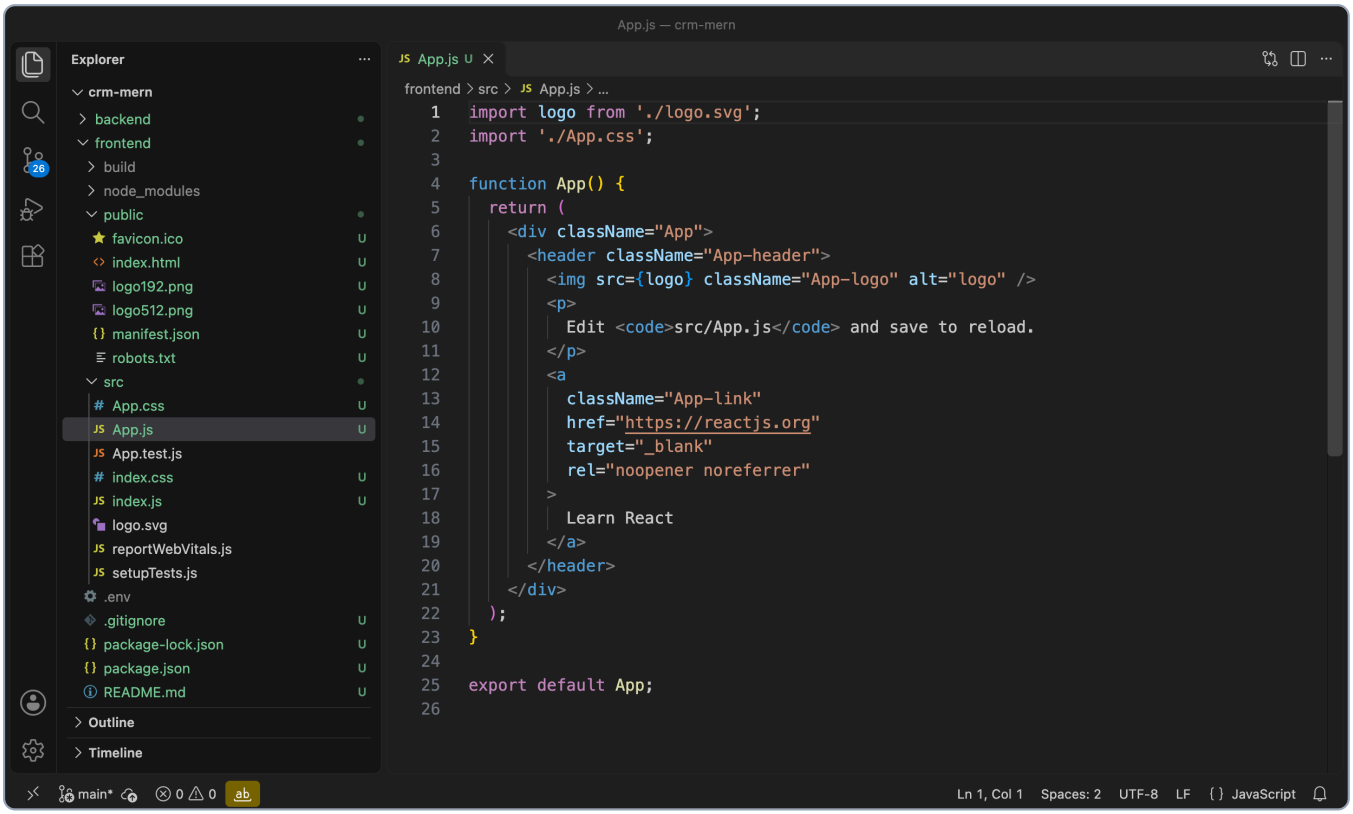
L'installation prend quelques minutes. Le navigateur s'ouvre ensuite sur **http://localhost:3000**.



CAPTURE RÉELLE La page de démonstration de Create React App : le logo React tourne.

> **À savoir :** Create React App n'est plus recommandé par l'équipe React pour les nouveaux projets professionnels. Il reste très pratique pour apprendre, et il a fonctionné pendant le test de ce TP. Si npm affiche des « vulnerabilities » à l'installation, ignore-les pour ce TP : **ne lance pas** `npm audit fix --force`, car cela peut casser le projet.

Étape 2 : visiter les fichiers



CAPTURE RÉELLE Les fichiers créés par Create React App, ici avec App.js ouvert.

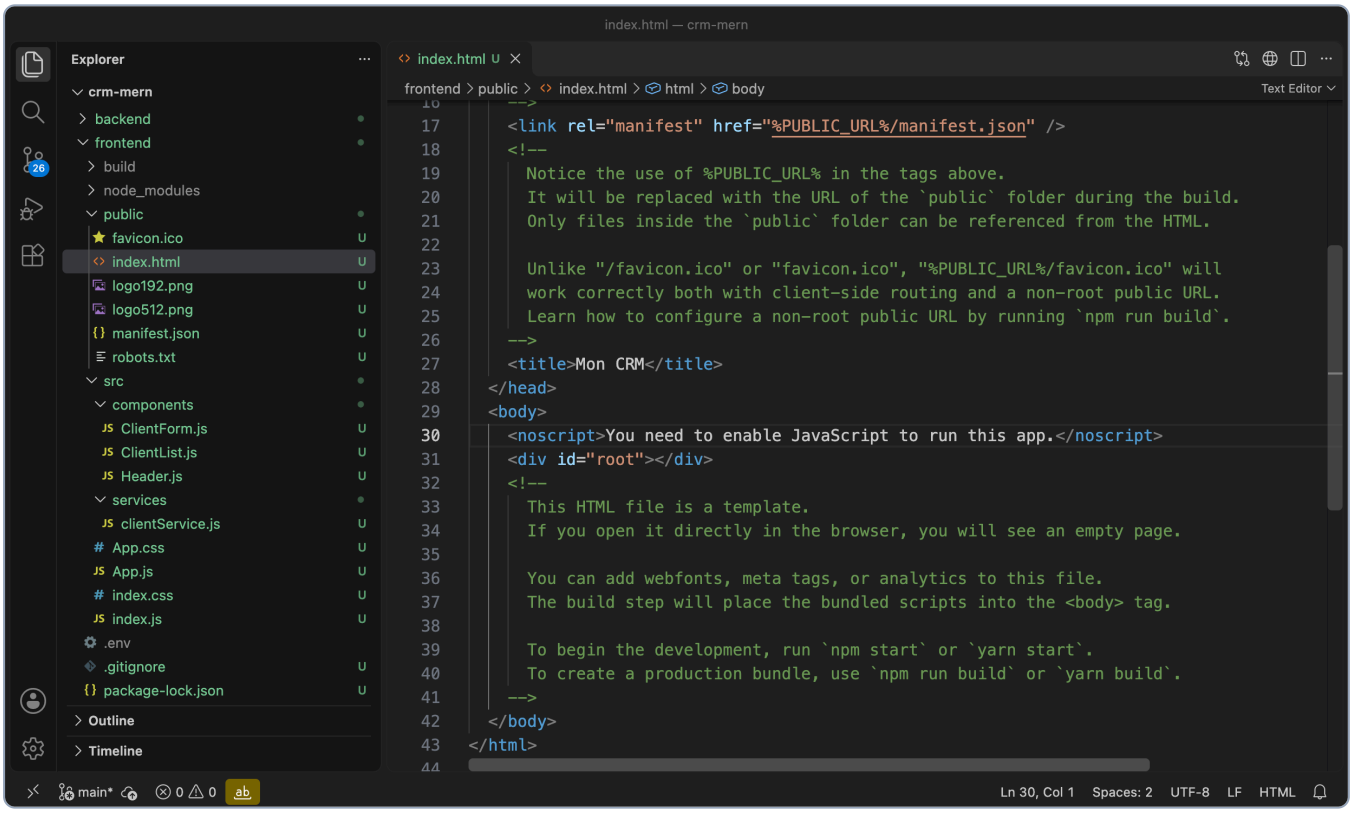
Fichier	Son rôle	On le garde ?
public/index.html	L'unique page HTML, avec la div « root »	**Oui, fichier principal**
src/index.js	Le point de départ : démarre React	**Oui, fichier principal**
src/App.js	Le composant principal : notre écran	**Oui, fichier principal**
src/index.css, src/App.css	Le style	**Oui**
package.json	Les dépendances et les commandes	**Oui**
src/App.test.js	Un test automatique	**Non, à supprimer**
src/setupTests.js	Configuration des tests	**Non, à supprimer**
src/reportWebVitals.js	Mesure des performances	**Non, à supprimer**
src/logo.svg	Le logo qui tourne	**Non, à supprimer**

Comment React s'affiche : le fichier principal

SCHÉMA **Le chemin de l'affichage**

public/index.html
Contient une div vide :
↓
src/index.js
Prend cette div et lui demande d'afficher App
↓
src/App.js
Décrit ce qui s'affiche à l'écran

La preuve que c'est une SPA : voici le code de la page reçue par le navigateur. Le corps de la page ne contient presque rien, juste la div `root`. Tout le reste est construit par JavaScript !



CAPTURE RÉELLE public/index.html : la ligne est l'endroit où React affichera toute l'application.



CAPTURE RÉELLE La preuve : si on désactive JavaScript, la page reste vide et affiche seulement le message de index.html.

Étape 3 : nettoyer le projet

1. Dans VS Code, supprime les 4 fichiers marqués « à supprimer » (clic droit, **Delete**).

2. Le navigateur affiche alors une erreur. C'est normal !



CAPTURE RÉELLE L'erreur « Module not found » : index.js essaie encore d'importer un fichier qui n'existe plus.

Lis bien ce message, car tu le reverras souvent. Il dit quel fichier pose problème, et quel import est introuvable. Pour le corriger, remplace **tout** le contenu de **src/index.js** :

src/index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import "./index.css";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Ligne	Ce qu'elle fait
`import App from "./App"`	Récupère le composant App
`document.getElementById("root")`	Trouve la div vide de index.html
`root.render(...)`	Affiche App à l'intérieur
`React.StrictMode`	Un mode qui aide à repérer les erreurs pendant le développement

Dans **public/index.html**, change aussi le titre : `<title>Mon CRM</title>`.

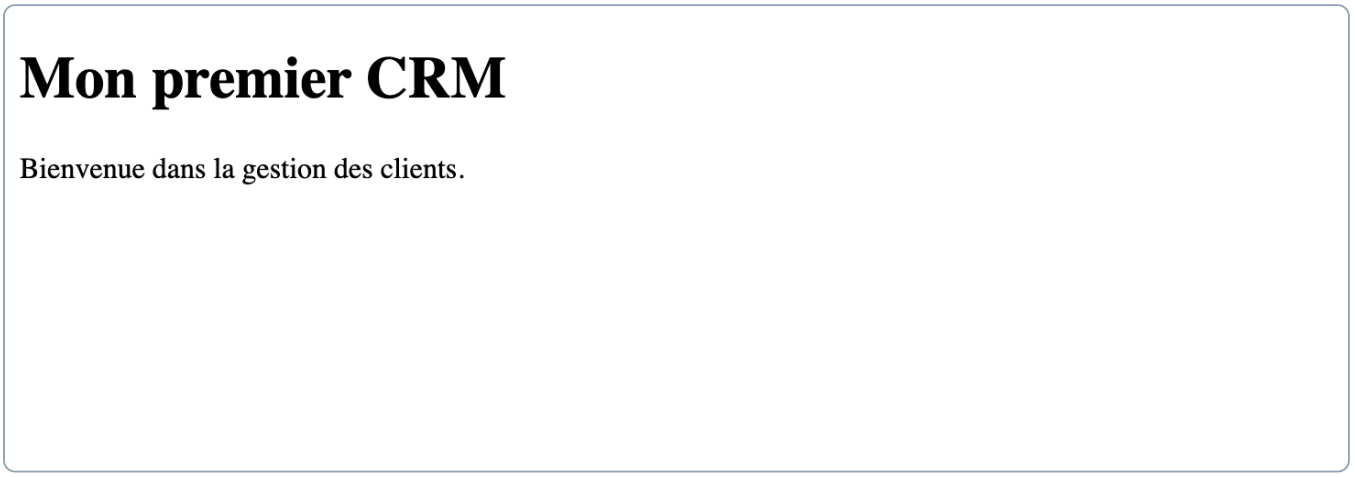
Étape 4 : ton premier composant

Un composant est simplement **une fonction qui retourne de l'affichage**. Remplace tout le contenu de **src/App.js** :

src/App.js

```
function App() {
  return (
    <div>
      <h1>Mon premier CRM</h1>
      <p>Bienvenue dans la gestion des clients.</p>
    </div>
  );
}

export default App;
```



CAPTURE RÉELLE Enregistre : le navigateur se met à jour tout seul, sans que tu l'actualises.

Cette écriture qui ressemble à du HTML dans du JavaScript s'appelle le **JSX**. Les règles à retenir :

- Le nom d'un composant commence par une **majuscule** : App, Header.
- Un composant retourne **un seul** élément parent, ici la `div`.
- On écrit `className` à la place de `class`.
- On ferme toutes les balises : `<input />`.
- `export default` permet d'utiliser le composant dans un autre fichier.

Étape 5 : mettre du JavaScript dans le JSX

Les **accolades { }** permettent d'insérer une valeur JavaScript dans l'affichage.

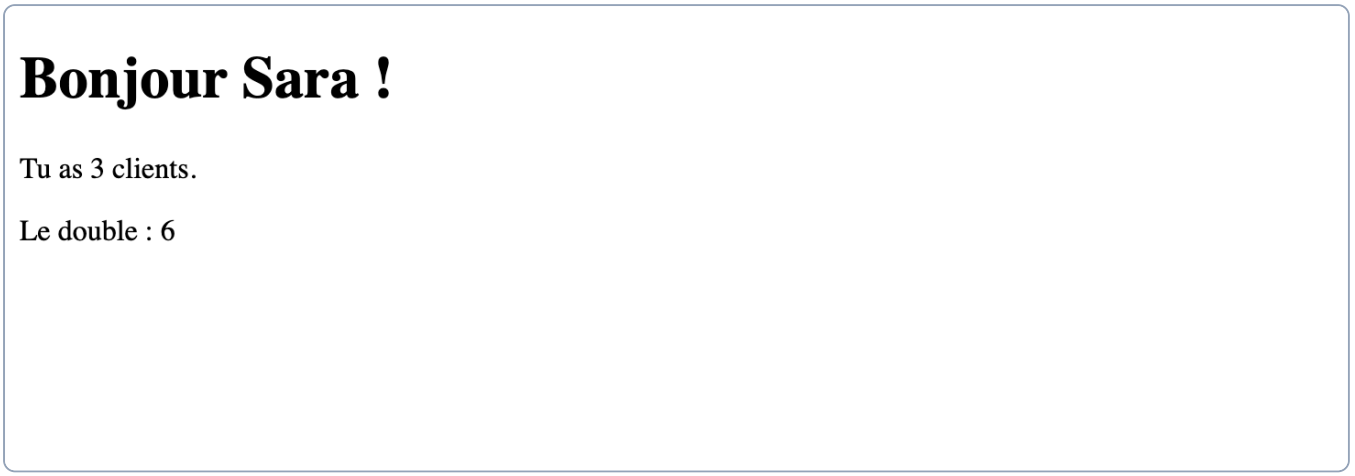
src/App.js

```
function App() {
  const prenom = "Sara";
  const nombreClients = 3;

  return (
    <div>
      <h1>Bonjour {prenom} !</h1>
      <p>Tu as {nombreClients} clients.</p>
      <p>Le double : {nombreClients * 2}</p>
```

```
    </div>
  );
}

export default App;
```



CAPTURE RÉELLE React remplace chaque { } par sa valeur et peut même faire un calcul.

Étape 6 : afficher une liste

`map()` parcourt un tableau et crée un élément pour chaque valeur. React demande une `key` unique pour reconnaître chaque ligne.

src/App.js

```
function App() {
  const clients = [
    { id: 1, nom: "Sara Amrani" },
    { id: 2, nom: "Yassine Bennani" },
    { id: 3, nom: "Lina Fassi" }
  ];

  return (
    <ul>
      {clients.map((client) => (
        <li key={client.id}>{client.nom}</li>
      ))}
    </ul>
  );
}

export default App;
```

- Sara Amrani
- Yassine Bennani
- Lina Fassi

CAPTURE RÉELLE Une ligne par client, sans écrire les 3 lignes à la main.

Étape 7 : créer un composant dans son propre fichier

Crée le dossier **src/components**, puis le fichier **Titre.js** :

src/components/Titre.js

```
function Titre() {
  return <h1>Mon CRM</h1>;
}

export default Titre;
```

Puis utilise-le dans App, comme une balise :

src/App.js

```
import Titre from "./components/Titre";

function App() {
  return (
    <div>
      <Titre />
      <p>Mon premier composant fonctionne !</p>
    </div>
  );
}

export default App;
```



CAPTURE RÉELLE Le composant Titre est affiché à l'intérieur de App.

Étape 8 : les props, pour transmettre des informations

Les **props** sont des informations envoyées d'un composant parent à un composant enfant, comme les attributs d'une balise HTML. Le même composant peut ainsi afficher des données différentes.

src/components/CarteClient.js

```
function CarteClient({ nom, entreprise }) {
  return (
    <div>
      <h3>{nom}</h3>
      <p>Entreprise : {entreprise}</p>
    </div>
  );
}

export default CarteClient;
```

src/App.js

```
import CarteClient from "./components/CarteClient";

function App() {
  return (
    <div>
      <h1>Mes clients</h1>
      <CarteClient nom="Sara Amrani" entreprise="Atlas Digital" />
      <CarteClient nom="Yassine Bennani" entreprise="Bennani Conseil" />
    </div>
  );
}

export default App;
```



CAPTURE RÉELLE Un seul composant CarteClient, utilisé deux fois avec des props différentes.

Étape 9 : useState, la mémoire du composant

Quand une donnée doit changer à l'écran, on utilise un **état** avec `useState`. Dès que l'état change, React met l'écran à jour.

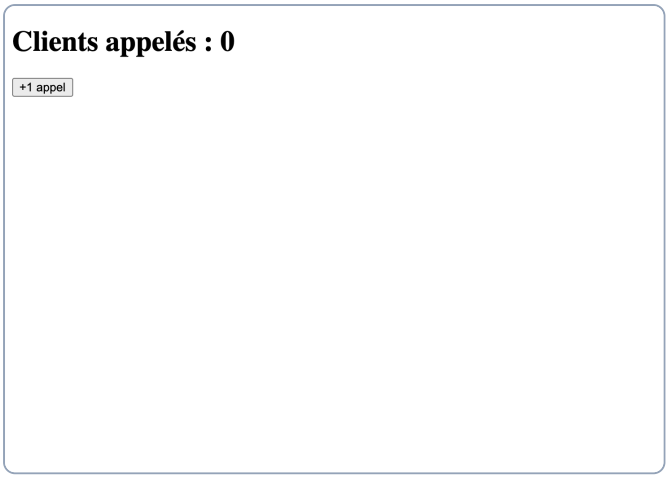
src/App.js

```
import { useState } from "react";

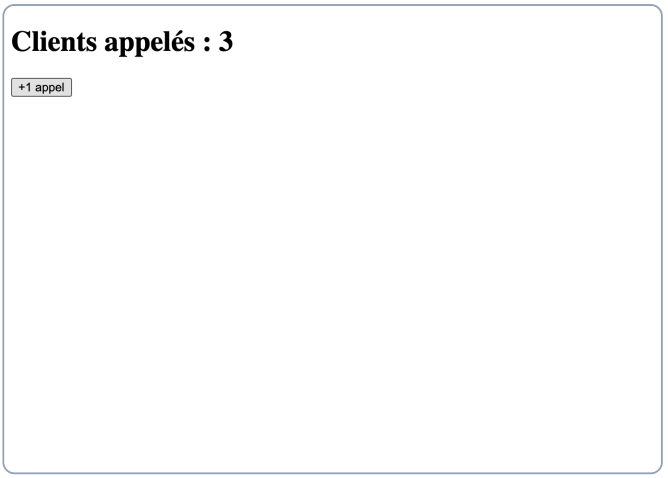
function App() {
  const [compteur, setCompteur] = useState(0);

  return (
    <div>
      <h1>Clients appelés : {compteur}</h1>
      <button onClick={() => setCompteur(compteur + 1)}>
        +1 appel
      </button>
    </div>
  );
}

export default App;
```



CAPTURE RÉELLE Au départ : 0.



CAPTURE RÉELLE Après 3 clics : 3, sans rechargement.

Élément	Signification
`useState(0)`	Crée l'état avec la valeur de départ 0
`compteur`	La valeur actuelle
`setCompteur`	La fonction qui change la valeur et met à jour l'écran
`onClick`	Le code à exécuter lors du clic

> **Erreur classique :** écrire `compteur = compteur + 1`. React ne saurait pas qu'il doit mettre l'écran à jour. Utilise toujours `setCompteur`.

Étape 10 : un champ de formulaire

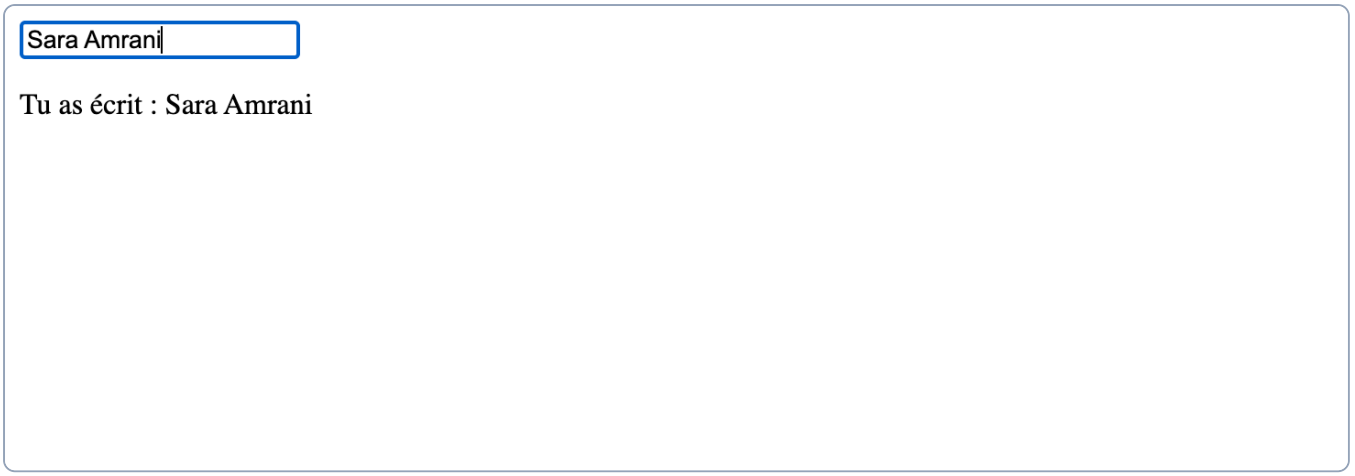
src/App.js

```
import { useState } from "react";

function App() {
  const [nom, setNom] = useState("");

  return (
    <div>
      <input
        placeholder="Tape un nom"
        value={nom}
        onChange={(event) => setNom(event.target.value)}
      />
      <p>Tu as écrit : {nom}</p>
    </div>
  );
}

export default App;
```



CAPTURE RÉELLE Chaque lettre tapée met à jour l'état, et donc le texte affiché.

`onChange` s'exécute à chaque frappe. `event.target.value` contient le texte du champ. `value={nom}` relie le champ à l'état.

Étape 11 : useEffect, pour charger des données

`useEffect` exécute du code **après l'affichage**. Avec le tableau vide `[]` à la fin, il ne s'exécute qu'une seule fois, au démarrage : c'est le bon moment pour demander les clients à l'API.

src/App.js

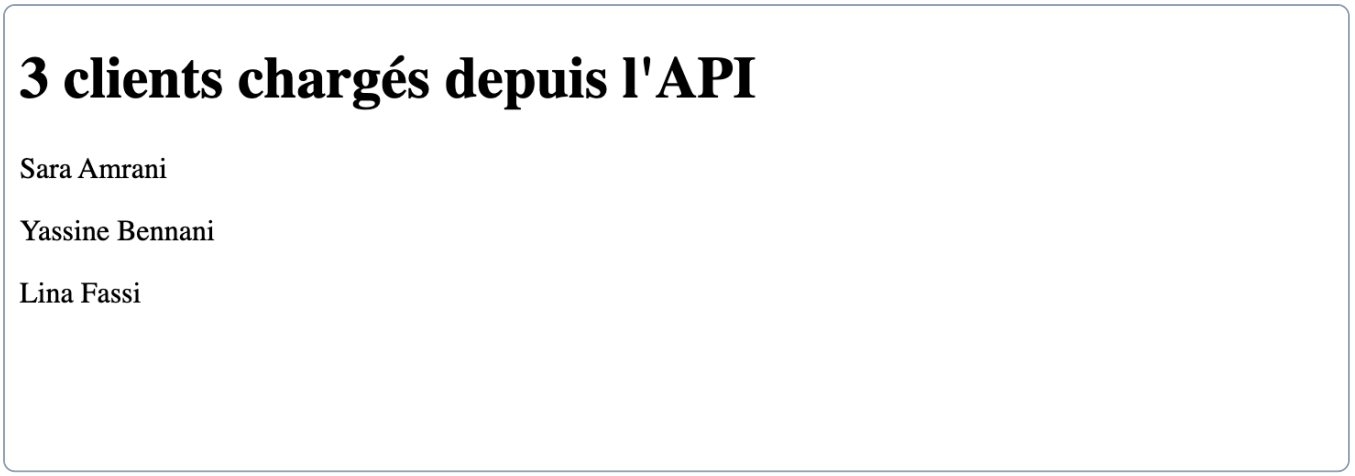
```
import { useEffect, useState } from "react";

function App() {
  const [clients, setClients] = useState([]);

  useEffect(() => {
    fetch("http://localhost:4000/api/clients")
      .then((response) => response.json())
      .then((data) => setClients(data));
  }, []);

  return (
    <div>
      <h1>{clients.length} clients chargés depuis l'API</h1>
      {clients.map((client) => (
        <p key={client._id}>{client.nom}</p>
      ))}
    </div>
  );
}

export default App;
```



CAPTURE RÉELLE Pour la première fois, React affiche de vraies données venant de l'API et de la base !

Si tu n'as aucun client ou une erreur : vérifie que le backend tourne encore dans l'autre terminal et que tu as ajouté des clients avec Postman.

✅ **Point de contrôle :** sais-tu expliquer ce qu'est une SPA, un composant, une prop et un état ? Tu connais maintenant tout le React utile pour ce TP.

Niveau 8 — Construire l'interface du CRM

SCHÉMA **Les pièces de notre écran**



Étape 1 : indiquer l'adresse de l'API

Crée **frontend/.env**. Avec Create React App, le nom d'une variable doit commencer par `REACT_APP_`.

frontend/.env

```
REACT_APP_API_URL=http://localhost:4000/api/clients
```

> **Attention :** les variables `REACT_APP_` sont visibles par tout le monde dans le navigateur. N'y mets jamais de secret. L'URI MongoDB reste uniquement dans le backend.

Important : après avoir créé ou modifié ce fichier, arrête le frontend (**Ctrl+C**) et relance `npm start`.

Étape 2 : le service qui parle à l'API

Crée **src/services/clientService.js**. Tous les appels à l'API sont rangés ici.

src/services/clientService.js

```
// L'adresse de l'API vient du fichier .env (ou de Vercel en production)
const API_URL = process.env.REACT_APP_API_URL;

// Lire tous les clients
export async function getClients() {
  const response = await fetch(API_URL);
  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message);
  }
  return data;
}

// Créer un client
export async function createClient(client) {
  const response = await fetch(API_URL, {
    method: "POST",
```

```
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(client)
  });
  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message);
  }
  return data;
}

// Modifier un client
export async function updateClient(id, modifications) {
  const response = await fetch(`${API_URL}/${id}`, {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(modifications)
  });
  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message);
  }
  return data;
}

// Supprimer un client
export async function deleteClient(id) {
  const response = await fetch(`${API_URL}/${id}`, { method: "DELETE" });
  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message);
  }
  return data;
}
```

Élément	Signification
`fetch(adresse, options)`	Envoie une demande HTTP depuis le navigateur
`JSON.stringify(client)`	Transforme l'objet JavaScript en texte JSON pour l'envoyer
`response.json()`	Transforme le JSON reçu en objet JavaScript
`response.ok`	Vrai si le code de réponse indique un succès (200 à 299)
`throw new Error(...)`	Signale l'erreur pour qu'App l'affiche

Étape 3 : le composant Header

src/components/Header.js

```
function Header({ total }) {
  return (
    <header>
      <h1>Mon CRM</h1>
      <p>{total} client(s) enregistré(s)</p>
    </header>
  );
}

export default Header;
```

Étape 4 : le composant ClientForm

src/components/ClientForm.js

```
import { useState } from "react";

function ClientForm({ onAjouter }) {
  const [nom, setNom] = useState("");
  const [email, setEmail] = useState("");
  const [entreprise, setEntreprise] = useState("");

  async function handleSubmit(event) {
    event.preventDefault(); // empêche le rechargement de la page

    const reussi = await onAjouter({ nom, email, entreprise });

    if (reussi) {
      setNom("");
      setEmail("");
      setEntreprise("");
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        placeholder="Nom"
        value={nom}
        onChange={(event) => setNom(event.target.value)}
        required
      />
      <input
        type="email"
```

```
        placeholder="Email"
        value={email}
        onChange={(event) => setEmail(event.target.value)}
        required
      />
      <input
        placeholder="Entreprise"
        value={entreprise}
        onChange={(event) => setEntreprise(event.target.value)}
      />
      <button type="submit">Ajouter</button>
    </form>
  );
}

export default ClientForm;
```

`event.preventDefault()` empêche le navigateur de recharger la page à l'envoi du formulaire : nous sommes dans une SPA ! Le formulaire ne connaît ni l'API ni la base : il donne simplement les données à App grâce à la prop `onAjouter`.

Étape 5 : le composant ClientList

src/components/ClientList.js

```
function ClientList({ clients, onChangerStatut, onSupprimer }) {
  if (clients.length === 0) {
    return <p>Aucun client à afficher.</p>;
  }

  return (
    <table>
      <thead>
        <tr>
          <th>Nom</th>
          <th>Email</th>
          <th>Entreprise</th>
          <th>Statut</th>
          <th></th>
        </tr>
      </thead>
      <tbody>
        {clients.map((client) => (
          <tr key={client._id}>
            <td>{client.nom}</td>
            <td>{client.email}</td>
            <td>{client.entreprise}</td>
            <td>
```

```
              <select
                value={client.statut}
                onChange={(event) =>
                  onChangerStatut(client._id, event.target.value)
                }
              >
                <option value="prospect">Prospect</option>
                <option value="client">Client</option>
                <option value="inactif">Inactif</option>
              </select>
            </td>
            <td>
              <button onClick={() => onSupprimer(client._id)}>Supprimer</button>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}

export default ClientList;
```

Étape 6 : assembler le tout dans App

Remplace tout **src/App.js** :

src/App.js

```
import { useEffect, useState } from "react";
import Header from "./components/Header";
import ClientForm from "./components/ClientForm";
import ClientList from "./components/ClientList";
import {
  getClients,
  createClient,
  updateClient,
  deleteClient
} from "./services/clientService";
import "./App.css";

function App() {
  const [clients, setClients] = useState([]);
  const [recherche, setRecherche] = useState("");
  const [message, setMessage] = useState("Chargement...");

  // Au premier affichage : demander les clients à l'API
  useEffect(() => {
```

```
    getClients()
      .then((data) => {
        setClients(data);
        setMessage("");
      })
      .catch((error) => setMessage(error.message));
  }, []);

  async function ajouterClient(nouveauClient) {
    try {
      const clientCree = await createClient(nouveauClient);
      setClients([clientCree, ...clients]);
      setMessage("Client ajouté !");
      return true;
    } catch (error) {
      setMessage(error.message);
      return false;
    }
  }

  async function changerStatut(id, statut) {
    try {
      const clientModifie = await updateClient(id, { statut });
      setClients(clients.map((c) => (c._id === id ? clientModifie : c)));
      setMessage("Statut modifié !");
    } catch (error) {
      setMessage(error.message);
    }
  }

  async function supprimerClient(id) {
    if (!window.confirm("Supprimer ce client ?")) {
      return;
    }
    try {
      await deleteClient(id);
      setClients(clients.filter((c) => c._id !== id));
      setMessage("Client supprimé !");
    } catch (error) {
      setMessage(error.message);
    }
  }

  // On garde uniquement les clients qui correspondent à la recherche
  const clientsFiltres = clients.filter((c) =>
    (c.nom + " " + c.email).toLowerCase().includes(recherche.toLowerCase())
  );
```

```
  return (
    <div className="container">
      <Header total={clients.length} />
      <ClientForm onAjouter={ajouterClient} />
      {message && <p className="message">{message}</p>}
      <input
        className="recherche"
        placeholder="Rechercher par nom ou email..."
        value={recherche}
        onChange={(event) => setRecherche(event.target.value)}
      />
      <ClientList
        clients={clientsFiltres}
        onChangerStatut={changerStatut}
        onSupprimer={supprimerClient}
      />
    </div>
  );
}

export default App;
```

Comprendre les 3 astuces sur les tableaux

Code	Ce qu'il fait
`[clientCree, ...clients]`	Nouvelle liste : le nouveau client, suivi des anciens
`clients.map(...)`	Nouvelle liste, en remplaçant le client modifié
`clients.filter(...)`	Nouvelle liste, en gardant seulement certains clients

On crée toujours une **nouvelle** liste, car React met l'écran à jour quand il reçoit une nouvelle valeur.

Étape 7 : le style

src/index.css

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f1f5f9;
  color: #1e293b;
}
```

src/App.css

```
.container {
  max-width: 900px;
```

```
  margin: 32px auto;
  padding: 24px;
  background: white;
  border-radius: 12px;
}

h1 {
  margin: 0;
  color: #1d4ed8;
}

form {
  display: flex;
  gap: 8px;
  margin: 20px 0;
}

input,
select,
button {
  padding: 10px;
  font-size: 14px;
  border: 1px solid #cbd5e1;
  border-radius: 6px;
}

form input {
  flex: 1;
}

button {
  background: #1d4ed8;
  color: white;
  border: none;
  cursor: pointer;
}

.message {
  padding: 10px;
  background: #dbeafe;
  border-radius: 6px;
}

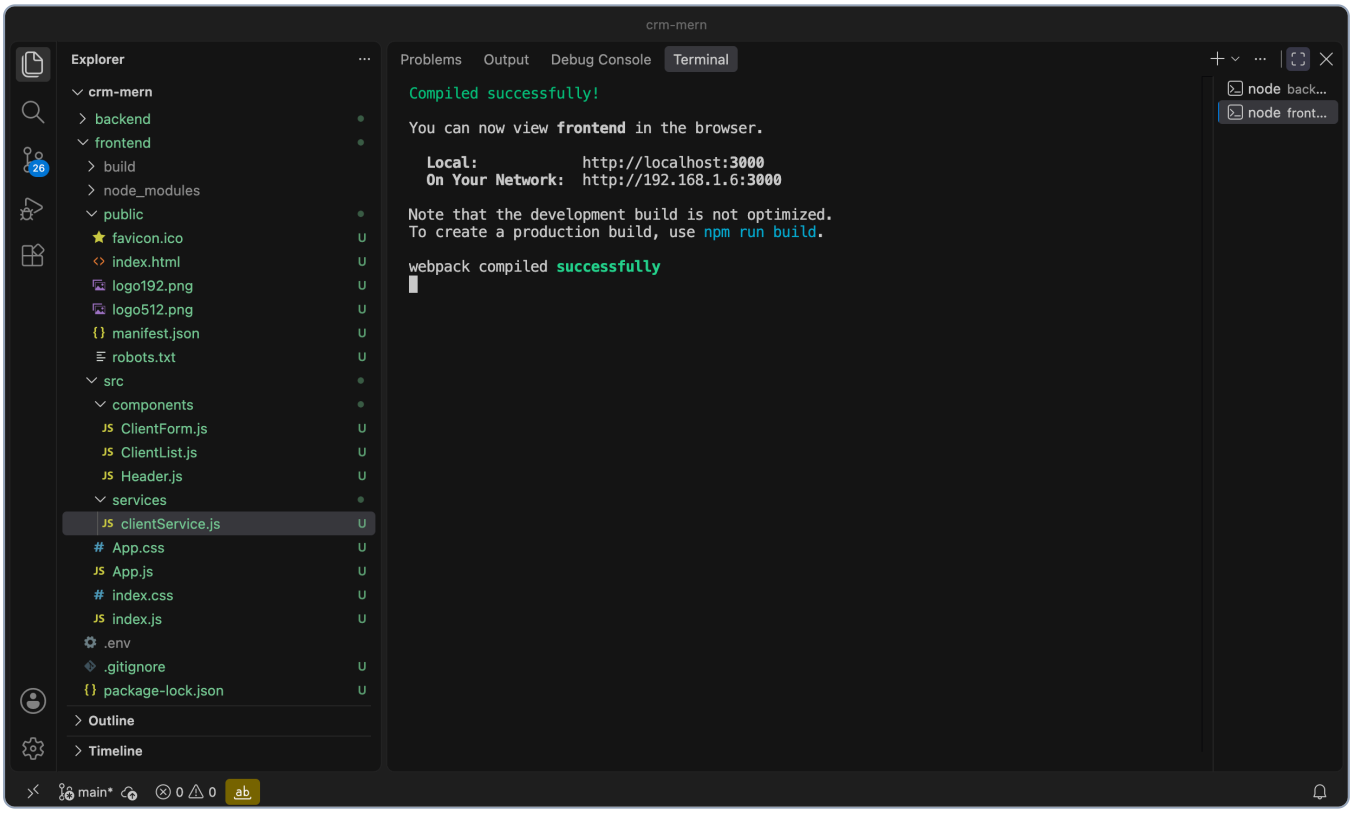
.recherche {
  width: 100%;
  box-sizing: border-box;
}
```

```css
table {
  width: 100%;
  margin-top: 16px;
  border-collapse: collapse;
}

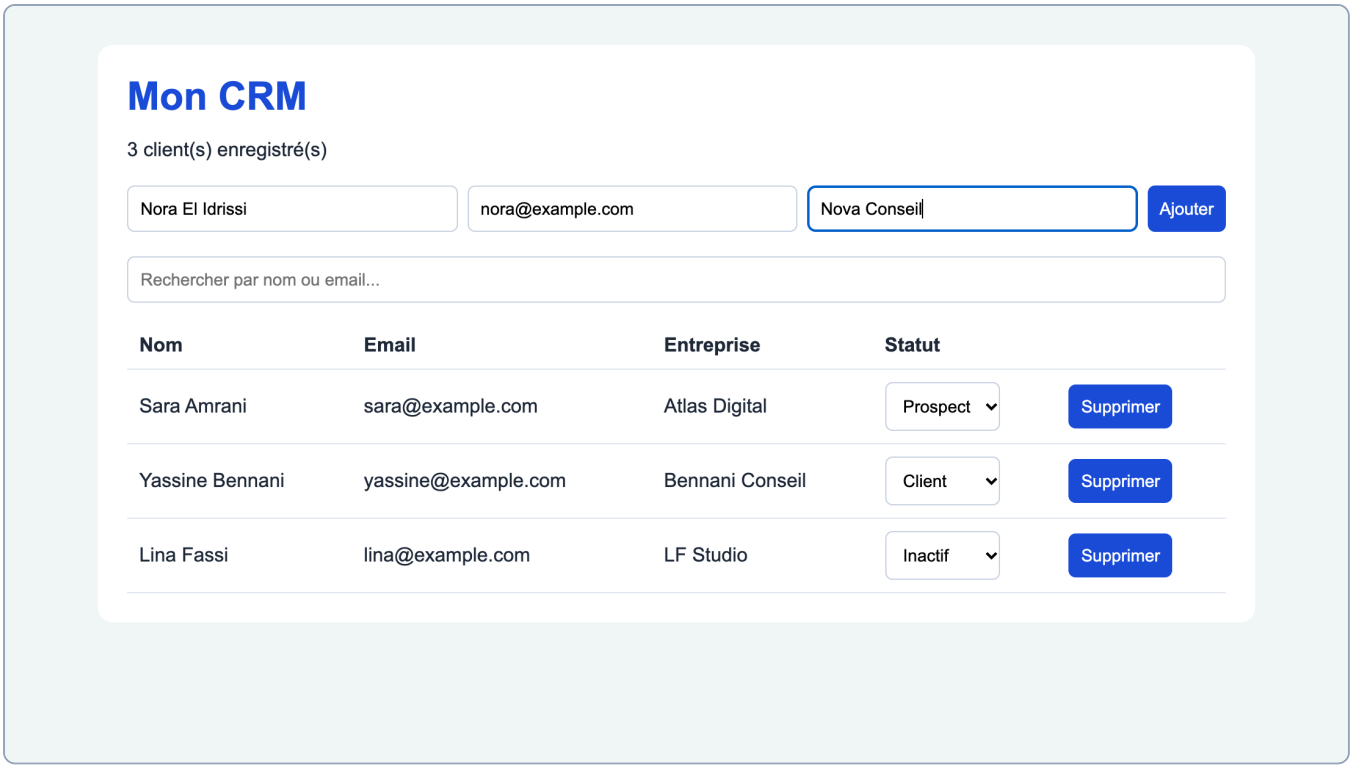
th,
td {
  padding: 10px;
  text-align: left;
  border-bottom: 1px solid #e2e8f0;
}
```

Étape 8 : tester le CRM complet

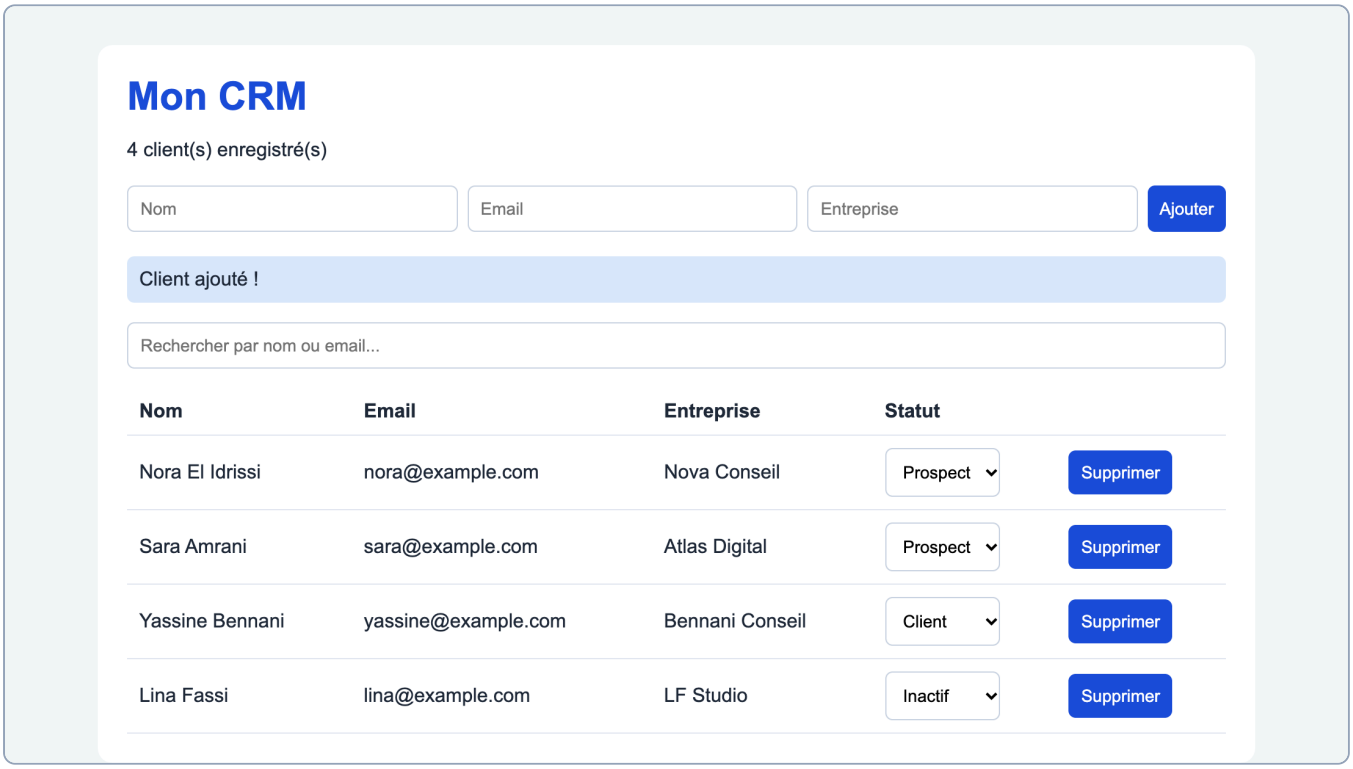
Deux terminaux doivent tourner : le backend et le frontend.



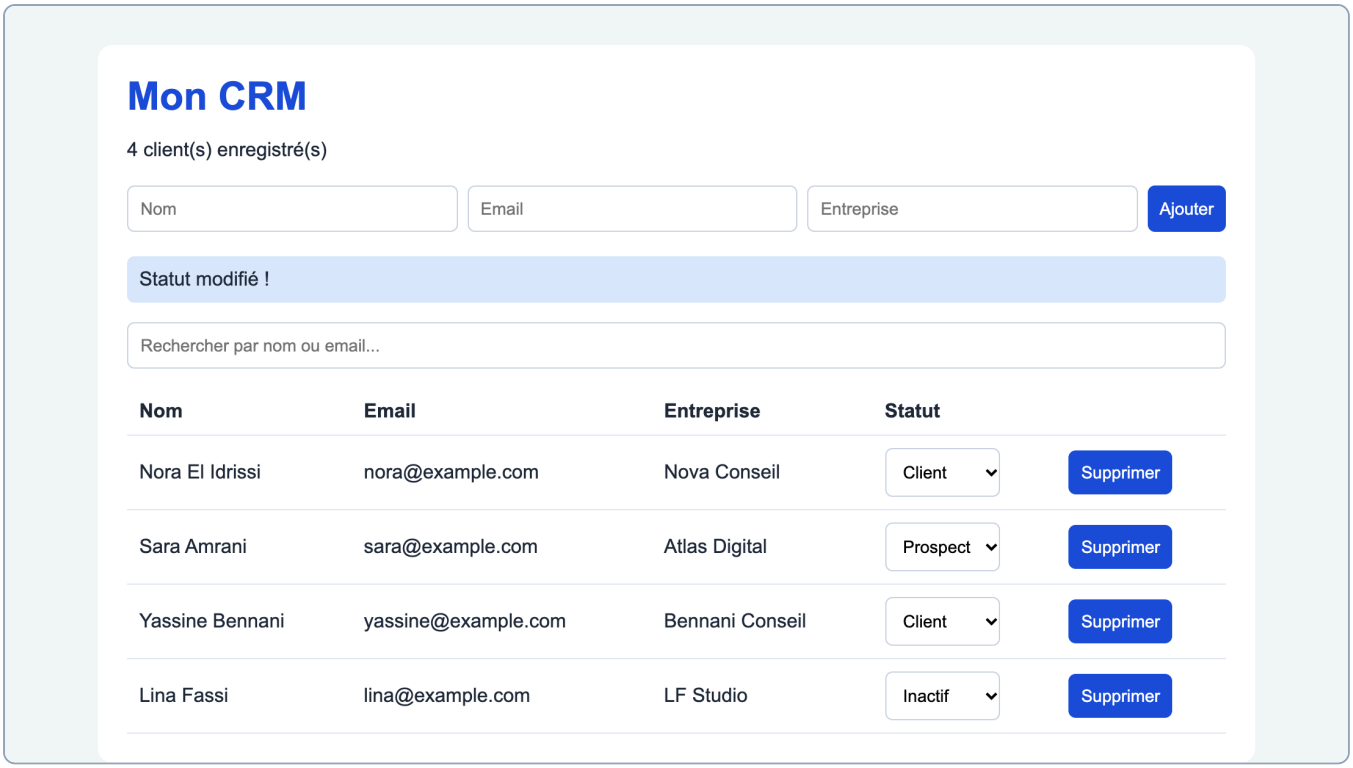
CAPTURE RÉELLE Le frontend compile sans erreur. L'onglet à droite permet de passer d'un terminal à l'autre.



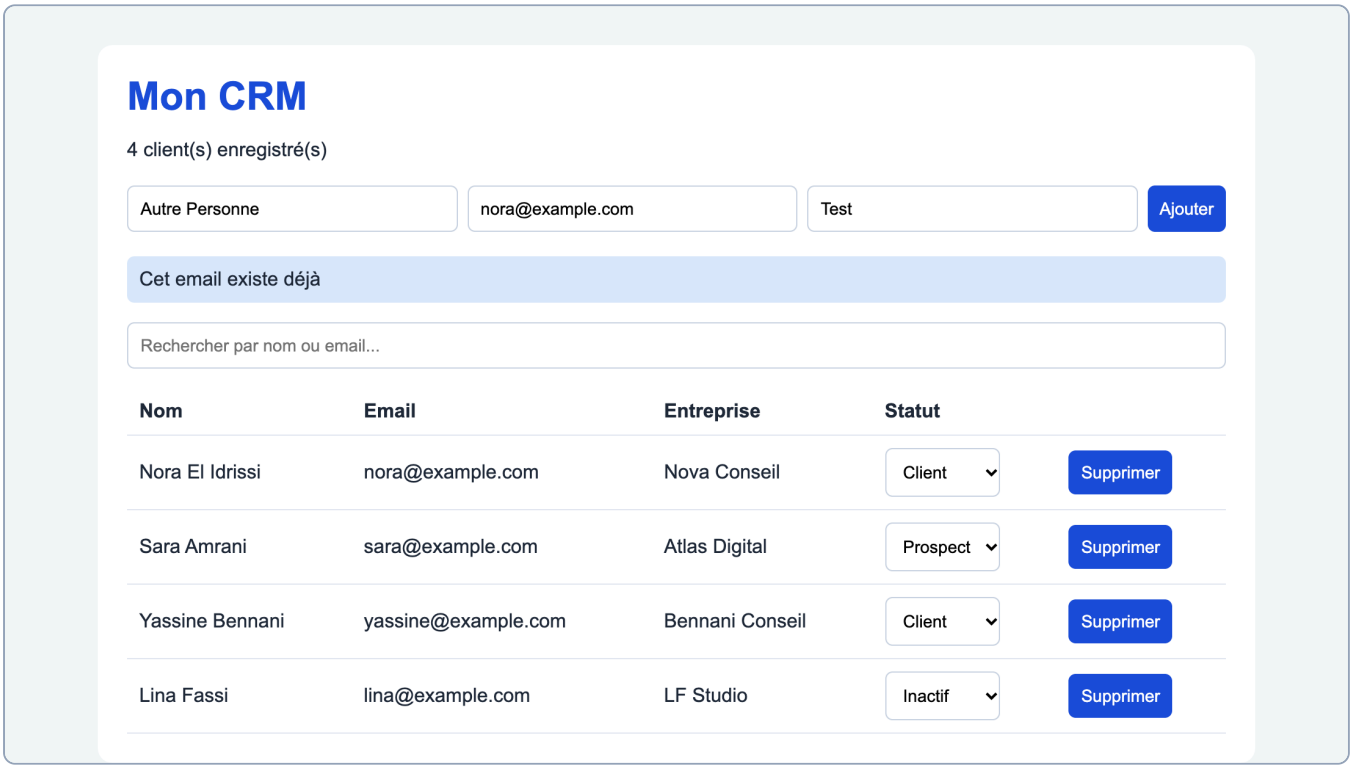
CAPTURE RÉELLE 1. Remplis le formulaire.



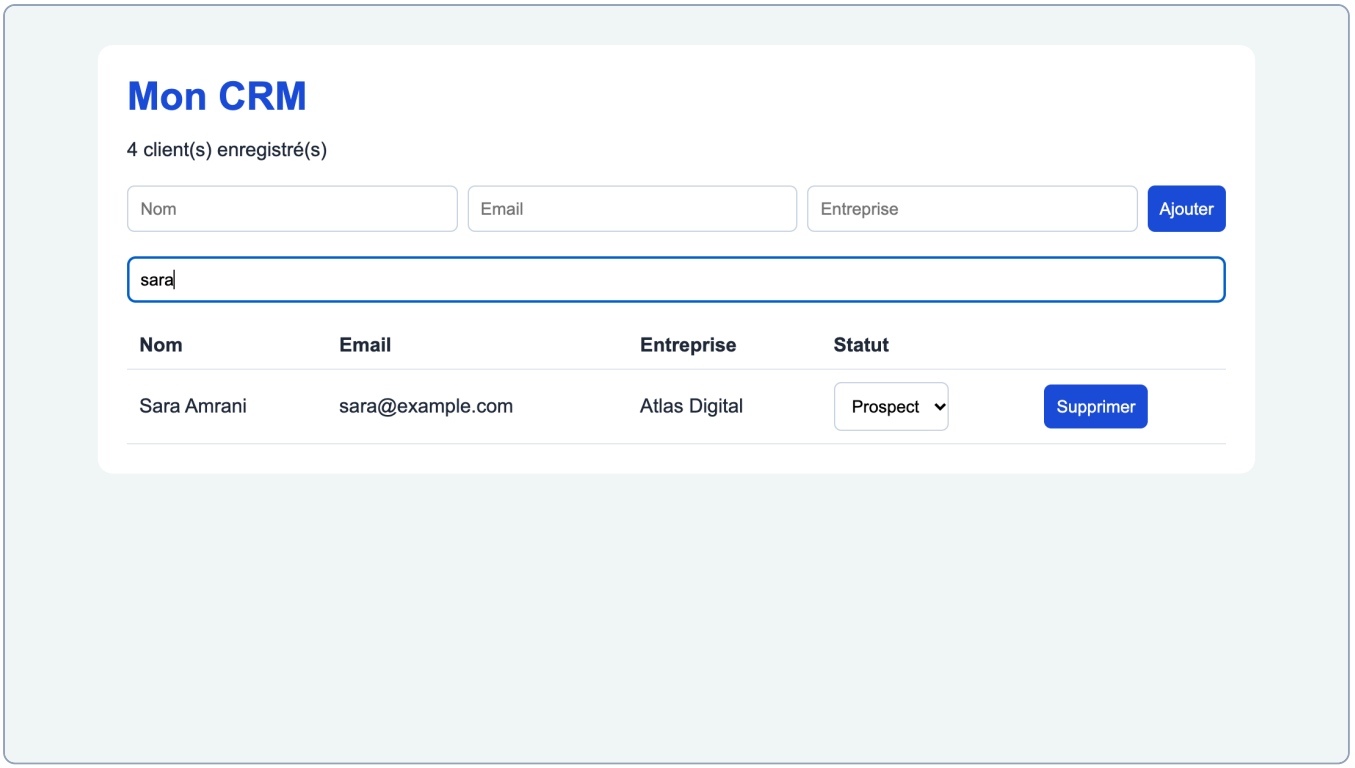
CAPTURE RÉELLE 2. Clique sur Ajouter : le client apparaît sans recharger la page.



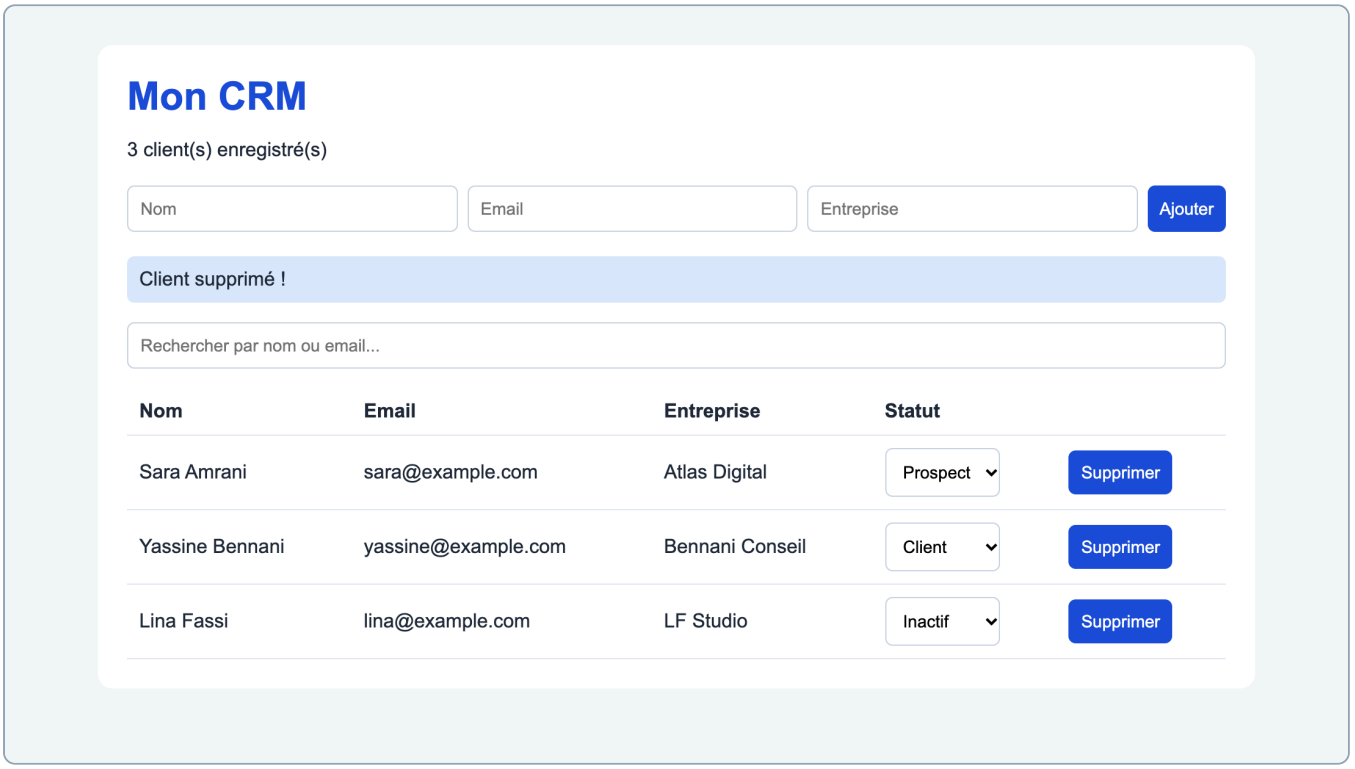
CAPTURE RÉELLE 3. Change le statut : la modification est enregistrée dans la base.



CAPTURE RÉELLE 4. Essaie le même email : l'erreur 409 de l'API s'affiche à l'écran.



CAPTURE RÉELLE 5. Tape dans la recherche : la liste se filtre à chaque lettre.



CAPTURE RÉELLE 6. Supprime un client : après confirmation, il disparaît.

SCHÉMA **Le voyage complet d'un clic sur « Ajouter »**



Voir les échanges dans le navigateur

Appuie sur **F12**, ouvre l'onglet **Network**, puis ajoute un client. Tu verras apparaître la demande `clients`. Clique dessus pour lire son code et sa réponse. C'est le meilleur outil pour trouver une erreur !

> ✅ **Point de contrôle :** actualise la page, ou ferme et rouvre le navigateur : tes clients sont toujours là, car ils sont dans Atlas et non dans React.

Niveau 9 — Publier sur GitHub

Git enregistre l'historique sur ton ordinateur. **GitHub** héberge ce code en ligne. Ne les confonds pas !

SCHÉMA **Le parcours de ton code**

Tes fichiers
Ce que tu modifies dans VS Code

↓ `git add .`

Zone de préparation
Ce qui fera partie de la prochaine sauvegarde

↓ `git commit -m "message"`

Historique local
Un point de sauvegarde sur ton ordinateur

↓ `git push`

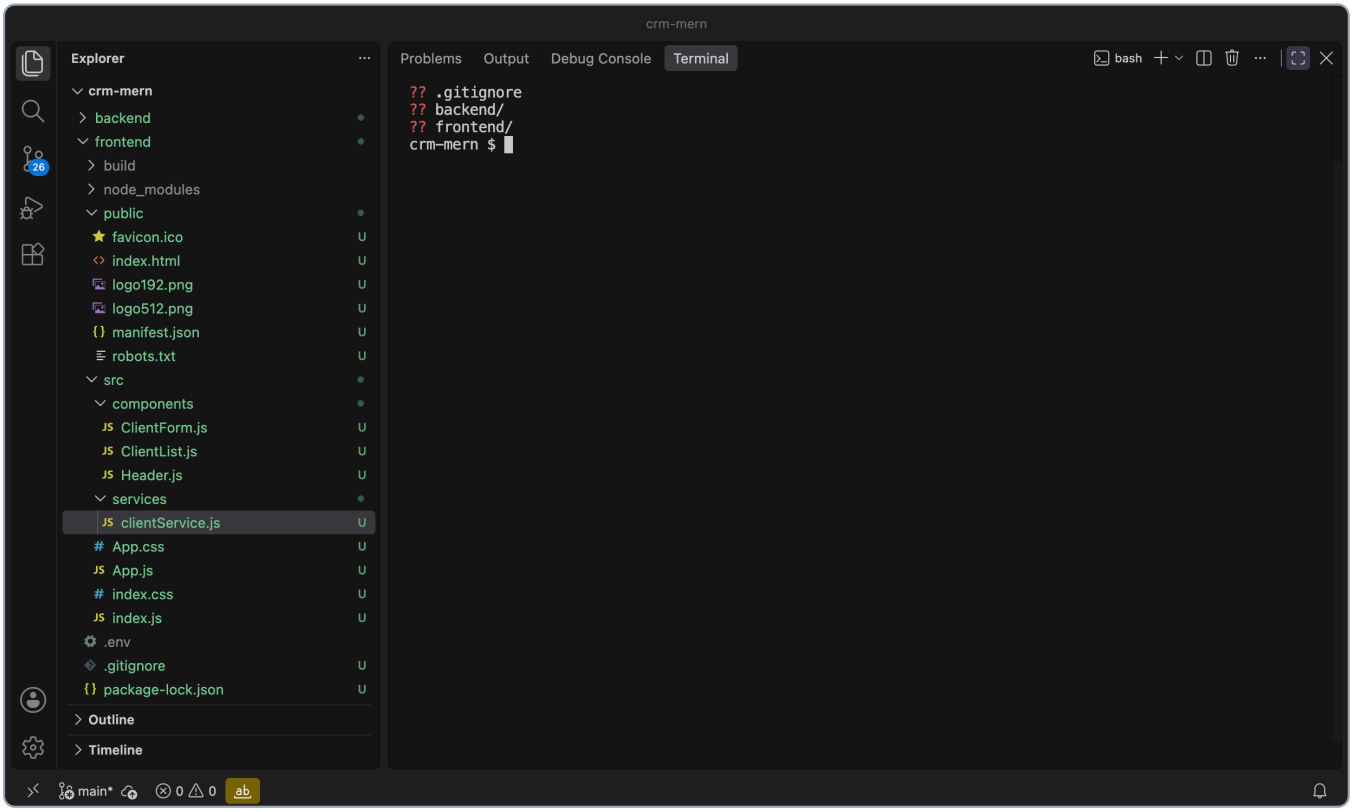
GitHub
Ton code est en ligne, et Vercel peut le récupérer

Étape 1 : vérifier AVANT d'envoyer

Dans un terminal, **à la racine crm-mern** :

Terminal — dans crm-mern

```
git status
```



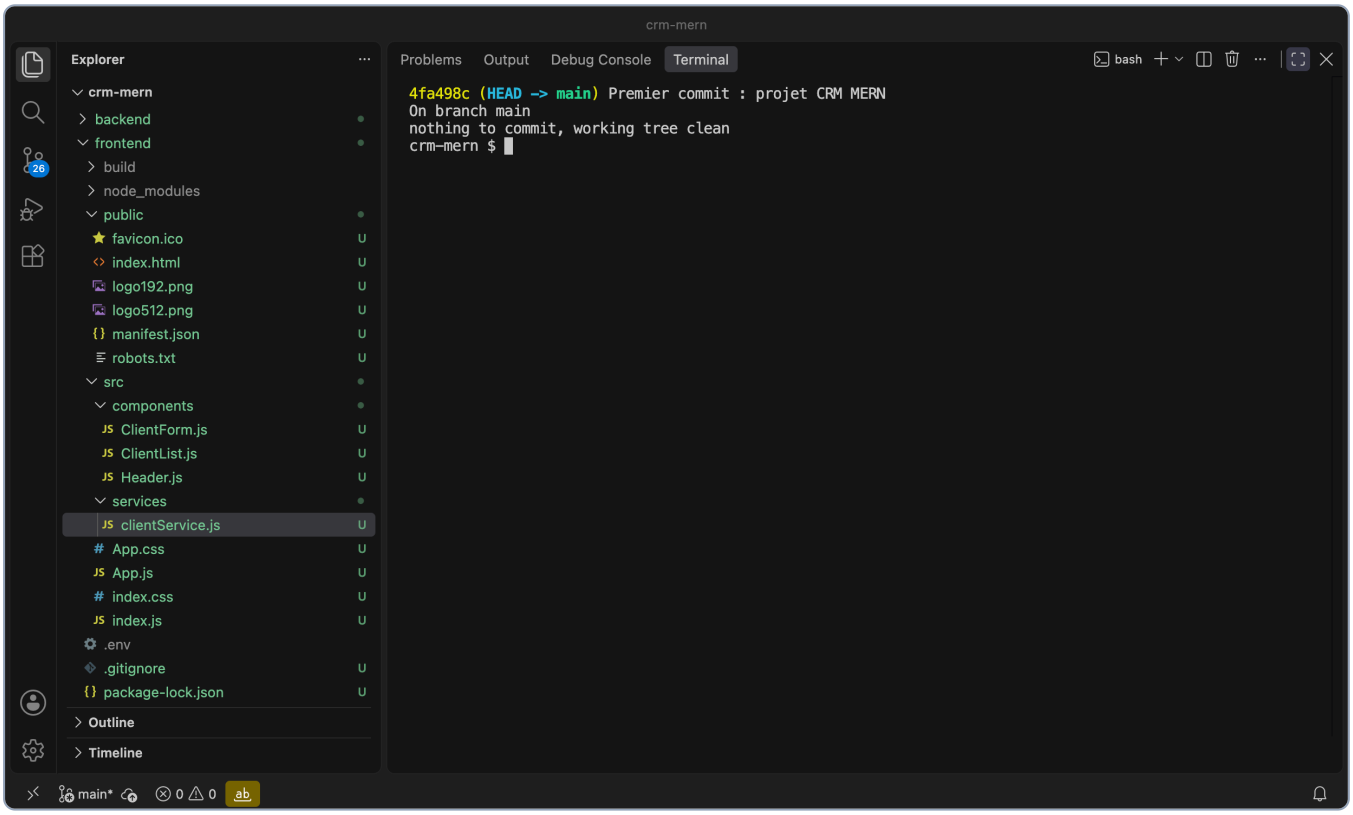
CAPTURE RÉELLE Git voit .gitignore, backend et frontend. Les « ?? » désignent des éléments pas encore suivis.

> **Contrôle de sécurité :** après le `git add .`, lance `git ls-files | grep env` (sur Windows PowerShell : `git ls-files | findstr env`). Tu dois voir **backend/.env.example**, mais **jamais backend/.env**. Si ton .env a déjà été publié, change immédiatement ton mot de passe sur Atlas.

Étape 2 : créer le premier commit

Terminal — dans crm-mern

```
git config user.name "Ton Nom"
git config user.email "ton-email@example.com"
git add .
git commit -m "Premier commit : projet CRM MERN"
git log --oneline
```



CAPTURE RÉELLE Le commit est créé, et Git annonce qu'il n'y a plus rien à enregistrer.

Étape 3 : créer le dépôt sur GitHub

1. Sur github.com, clique sur **+** en haut à droite, puis **New repository**.
2. Repository name : **crm-mern**.
3. Choisis **Public** ou **Private** selon la consigne.
4. **Ne coche pas** l'ajout d'un README : ton projet existe déjà.
5. Clique sur **Create repository** et copie l'adresse du dépôt.

Étape 4 : envoyer le code

Terminal — dans crm-mern

```
git remote add origin https://github.com/TON-COMPTE/crm-mern.git
git push -u origin main
```

À la première fois, une fenêtre te demande de te connecter à GitHub. Accepte. GitHub n'accepte pas ton mot de passe de site dans le terminal : s'il t'en demande un, utilise la connexion par le navigateur ou un jeton (*Personal Access Token*).

Après chaque évolution

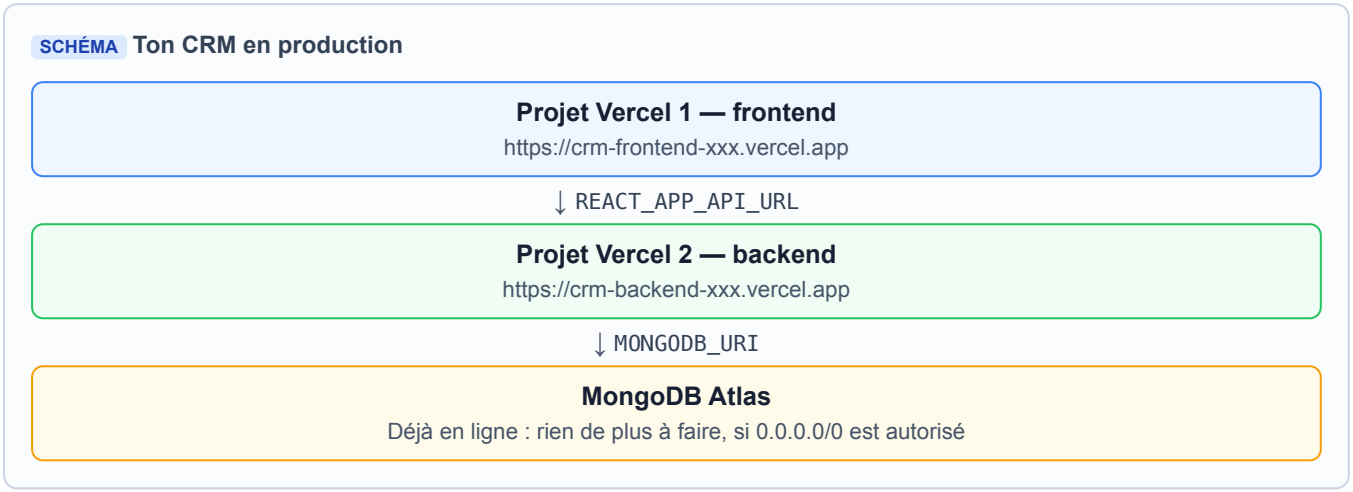
Les 3 commandes à répéter

```
git add .
git commit -m "Ajout de la recherche"
git push
```

✅ **Point de contrôle :** sur GitHub, tu vois les dossiers **backend** et **frontend**. Ouvre **frontend** : il n'est pas vide. Ouvre **backend** : il n'y a **pas** de fichier .env ni de dossier node_modules.

Niveau 10 — Déployer sur Vercel

Déployer, c'est mettre ton application sur un serveur pour que tout le monde puisse l'utiliser. **Vercel** récupère ton code sur GitHub et le publie. Nous allons créer **2 projets Vercel** à partir du **même dépôt GitHub**.



> **L'ordre compte :** 1. le backend → 2. le frontend → 3. retour au backend pour autoriser l'adresse du frontend. En effet, chacun a besoin de connaître l'adresse de l'autre.

Partie 1 : déployer le backend

1. Sur vercel.com, clique sur **Add New**, puis **Project**.
2. Importe ton dépôt **crm-mern** avec **Import**. S'il n'apparaît pas, autorise l'accès de Vercel à ce dépôt.
3. Project Name : **crm-backend**.
4. À la ligne **Root Directory**, clique sur **Edit** et choisis **backend**. C'est **l'étape à ne pas oublier**.
5. Ouvre **Environment Variables** et ajoute la variable ci-dessous.
6. Clique sur **Deploy**.

Nom	Valeur
MONGODB_URI	Ton URI Atlas complète, avec /crm

Vercel reconnaît Express tout seul, grâce à **server.js** : aucune configuration en plus. Il n'y a pas de variable PORT à ajouter.

Tester le backend en ligne

Quand c'est terminé, copie l'adresse donnée par Vercel et ouvre-la dans ton navigateur, en ajoutant `/api/clients` :

Test dans le navigateur

```
https://crm-backend-xxx.vercel.app/
→ {"message":"API CRM opérationnelle"}
```

```
https://crm-backend-xxx.vercel.app/api/clients
→ la liste de tes clients en JSON
```

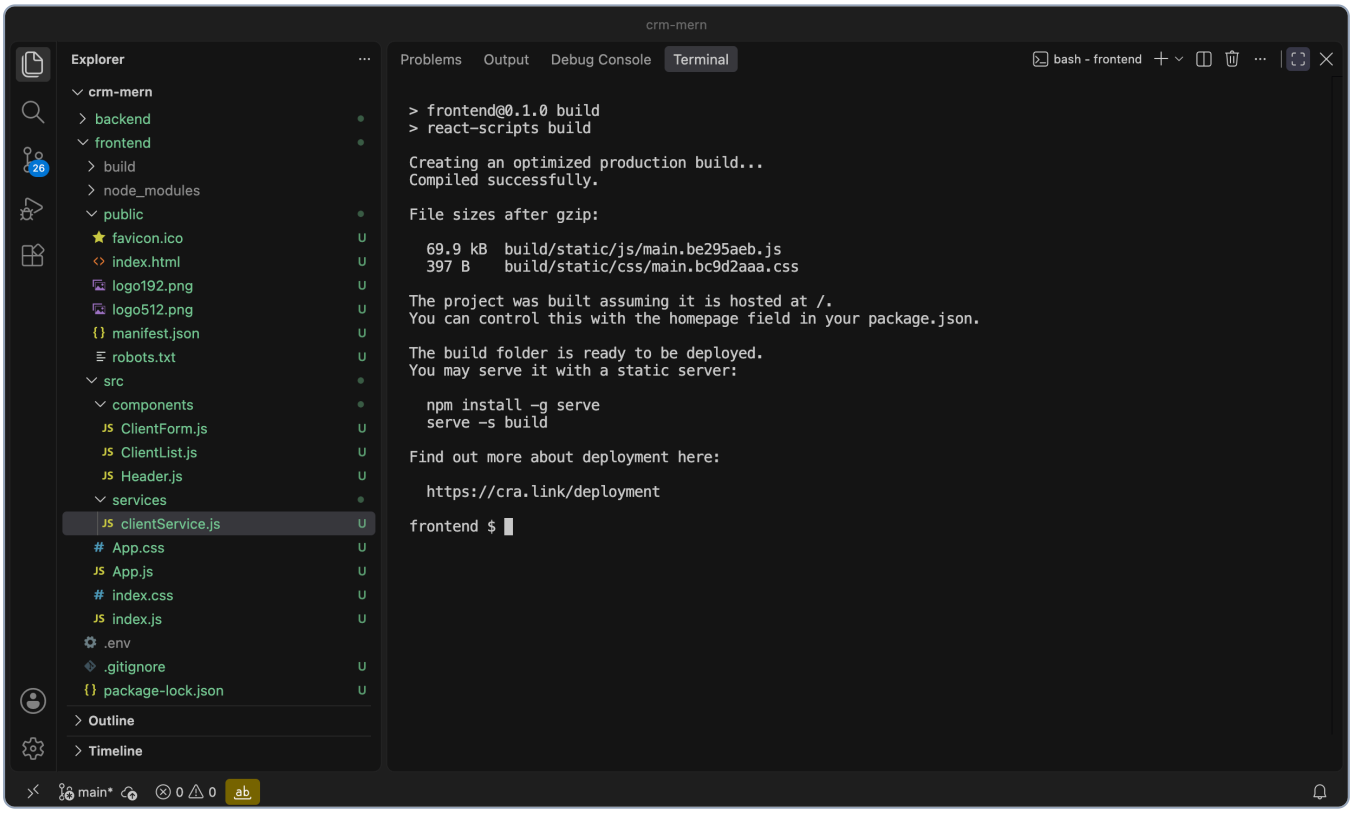
> **Ne passe pas au frontend tant que ce test échoue.** En cas d'erreur, ouvre l'onglet **Logs** du projet Vercel. Le plus souvent, il s'agit de : MONGODB_URI mal copiée, mot de passe incorrect, ou 0.0.0.0/0 absent sur Atlas.

Partie 2 : déployer le frontend

1. Reviens au tableau de bord : **Add New**, **Project**, et importe **le même dépôt**.
2. Project Name : **crm-frontend**.
3. Root Directory : **Edit**, puis choisis **frontend**.
4. Vérifie que le framework est bien détecté (**Create React App**). Commande de build : `npm run build`. Dossier de sortie : `build`.
5. Ajoute la variable ci-dessous, puis clique sur **Deploy**.

Nom	Valeur (ton adresse backend + /api/clients)
REACT_APP_API_URL	https://crm-backend-xxx.vercel.app/api/clients

Pendant le déploiement, Vercel lance `npm run build`, qui fabrique une version optimisée du site :



CAPTURE RÉELLE Le même build, testé ici dans le terminal : « Compiled successfully » et « The build folder is ready to be deployed ».

Partie 3 : autoriser le frontend (CORS)

À ce stade, ton site s'ouvre, mais il affiche une erreur. C'est normal : le backend n'autorise pas encore l'adresse du frontend.

1. Copie l'adresse de production de ton frontend, **sans « / » à la fin**.
2. Ouvre le projet **crm-backend**, puis **Settings**, puis **Environment Variables**.
3. Ajoute la variable ci-dessous et clique sur **Save**.
4. Ouvre l'onglet **Deployments**, clique sur ⋯ sur le dernier déploiement, puis sur **Redeploy**.

Nom	Valeur
FRONTEND_URL	https://crm-frontend-xxx.vercel.app

À retenir : une variable modifiée sur Vercel ne s'applique qu'à un **nouveau déploiement**. Après chaque changement, il faut redéployer.

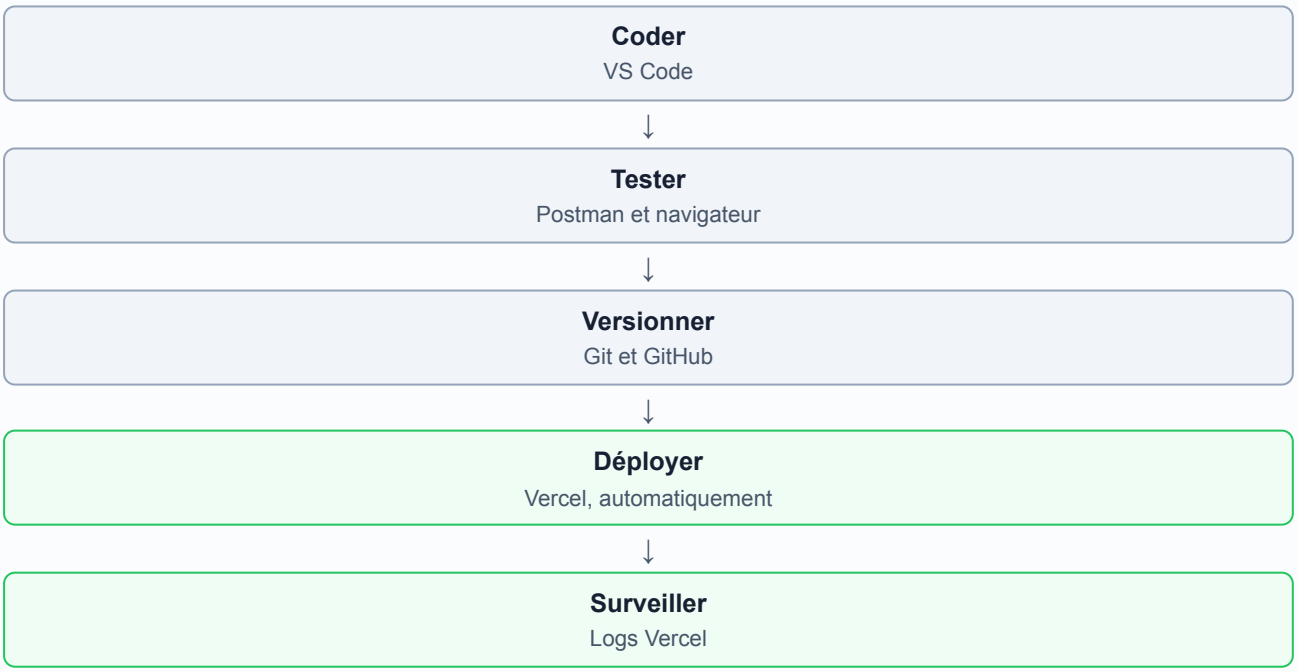
Partie 4 : la victoire

1. Ouvre l'adresse du frontend, sur ton téléphone par exemple.
2. Ajoute un client, change son statut, actualise la page.
3. Vérifie le client dans le Data Explorer d'Atlas.

Le déploiement automatique (CI/CD)

Dès maintenant, chaque `git push` sur la branche main relance le déploiement. Essaie : change le titre du CRM dans Header.js, puis fais add, commit et push. Le site se met à jour tout seul !

SCHÉMA **Le cycle DevOps**



✅ **Point de contrôle :** ton CRM fonctionne en ligne, depuis un autre appareil que ton ordinateur. **Bravo, tu as créé et déployé une application full-stack !**

Boîte de secours — Les erreurs fréquentes

Ce que tu vois	La cause probable	La solution
Cannot find module	Paquet non installé ou mauvais chemin	`npm install` dans le bon dossier ; vérifie le chemin de l'import
Module not found dans React	Import d'un fichier supprimé	Enlève ou corrige l'import
Page React blanche	Erreur JavaScript	F12, onglet Console
address already in use	Le port est déjà utilisé	Ferme l'autre serveur (Ctrl+C dans son terminal)
Connexion à la base impossible	URI, mot de passe ou IP	Vérifie .env et IP Access List, puis redémarre
bad auth / Authentication failed	Mauvais utilisateur de base	Vérifie l'utilisateur de base, pas ton compte Atlas
Données dans une base « test »	/crm oublié dans l'URI	Ajoute /crm avant le ?
Erreur CORS dans la console	FRONTEND_URL incorrecte	Adresse exacte, sans « / » final, puis Redeploy du backend
Failed to fetch	Backend arrêté ou mauvaise adresse	Vérifie le backend et REACT_APP_API_URL
Le site en ligne appelle encore localhost	Variable absente au moment du build	Ajoute REACT_APP_API_URL, puis Redeploy du frontend
Dossier frontend vide sur GitHub	Dépôt Git imbriqué	Supprime frontend/.git, puis `git rm --cached frontend`, `git add .`, commit et push
Vercel échoue à trouver le projet	Root Directory oublié	Settings : mets backend ou frontend

Méthode de diagnostic : 1. Console du navigateur (F12). 2. Onglet Network. 3. Terminal du backend, ou Logs sur Vercel. 4. Test de l'API avec Postman : cela indique si le problème vient du frontend ou du backend.

Boss final — Présente ton CRM

La démonstration (5 à 8 minutes)

N°	Action à montrer	Preuve attendue
1	Ouvrir le CRM en ligne	Adresse Vercel du frontend
2	Ajouter un prospect	Le client apparaît dans le tableau
3	Montrer la demande dans Network	POST et code 201
4	Montrer le document dans Atlas	Data Explorer
5	Changer son statut	Modification conservée après actualisation
6	Rechercher un client	Liste filtrée
7	Tester un email en double	Message d'erreur (409)
8	Supprimer le client	Disparition de la liste
9	Montrer Postman	Requêtes et codes HTTP
10	Montrer GitHub	Commits ; pas de .env

Les livrables

- Le lien du dépôt GitHub.
- Le lien Vercel du frontend et le lien Vercel de l'API.
- Des captures Postman : un POST 201 et une erreur 400 ou 409.
- Une capture du Data Explorer d'Atlas, sans afficher de mot de passe.
- Un court README : installation, lancement, variables nécessaires.

Barème indicatif sur 20

Critère	Points
MongoDB Atlas : cluster, utilisateur, accès réseau, données	3
API REST : routes CRUD et codes HTTP	4
Gestion des erreurs et tests Postman	2
React : composants, props, état, formulaire, appel à l'API	5
Git et GitHub : commits, README, aucun secret publié	2
Déploiement Vercel du frontend et du backend	2
Présentation et compréhension à l'oral	2
TOTAL	**20**

Les questions de l'oral

1. Que se passe-t-il entre le clic sur « Ajouter » et l'enregistrement dans la base ?
2. Qu'est-ce qu'une SPA ? Comment le prouver avec index.html ?
3. Quel est le rôle de index.html, index.js et App.js ?
4. Quelle est la différence entre une prop et un état ?
5. À quoi servent useState et useEffect ?
6. Quelle est la différence entre GET, POST, PATCH et DELETE ?
7. Quelle est la différence entre req.body et req.params ?
8. Pourquoi une création renvoie-t-elle 201, et un doublon 409 ?
9. Quelle est la différence entre un compte Atlas et un utilisateur de base ?
10. Pourquoi autoriser 0.0.0.0/0, et quel est le risque ?
11. Pourquoi le fichier .env ne va-t-il jamais sur GitHub ?
12. Pourquoi React ne se connecte-t-il pas directement à MongoDB ?
13. Quelle est la différence entre Git et GitHub ?
14. Pourquoi faut-il redéployer après avoir modifié une variable sur Vercel ?

Les bonus pour les plus rapides

- Ajouter le champ téléphone au formulaire (il existe déjà dans le modèle).
- Filtrer la liste par statut avec une liste déroulante.
- Afficher le nombre de prospects, de clients et d'inactifs.
- Modifier le nom d'un client depuis le tableau.

Fiche mémo

Les commandes

À garder sous la main

```
# Backend (dans backend/)
npm install        # installer les paquets
npm run dev        # lancer en développement

# Frontend (dans frontend/)
npm install
npm start          # lancer en développement
npm run build      # fabriquer la version de production

# Git (dans crm-mern/)
git status
git add .
git commit -m "message"
git push
```

Les variables d'environnement

Variable	Où	En local	Sur Vercel
MONGODB_URI	Backend	URI Atlas	URI Atlas
FRONTEND_URL	Backend	http://localhost:3000	Adresse du frontend Vercel
PORT	Backend	4000	À ne pas définir
REACT_APP_API_URL	Frontend	http://localhost:4000/api/clients	Adresse du backend Vercel + /api/clients

Le glossaire

Mot	Définition simple
API	Les demandes qu'on peut faire à un serveur
Endpoint	Une méthode et une adresse de l'API : GET /api/clients
JSON	Format texte pour échanger des données
Payload	Les données transportées dans une demande
CRUD	Créer, Lire, Modifier, Supprimer
Middleware	Une fonction Express exécutée avant la route, comme express.json()
Cluster	Les serveurs qui hébergent ta base sur Atlas
Collection / Document	Un groupe de documents / un enregistrement, ici un client
Schéma / Modèle	La description d'un client / l'outil Mongoose pour le manipuler
SPA	Application d'une seule page, mise à jour par JavaScript
Composant	Une fonction qui retourne de l'affichage
JSX	L'écriture proche du HTML utilisée dans React
Prop / État	Une information reçue du parent / la mémoire du composant
CORS	La règle qui dit quels sites peuvent appeler une API
Commit / Push	Une sauvegarde locale / son envoi sur GitHub
Build	La version optimisée du site, prête à être publiée
Déploiement	La mise en ligne de l'application
CI/CD	L'automatisation des vérifications et de la mise en ligne